Bash Shell Scripting & File Management

A Comprehensive Training Guide

From Absolute Beginner to Expert Mastery

```
╔═══════════════════════════════════════════════════════════════╗
║                                                               ║
║  ▀▀▀▀▀▀▄  ▀▀▀▀▀▄ ▀▀▀▀▀▀▄▄▄  ▄▄▄    ▀▀▀▀▀▀▄ ▀▀▀▀▀▀▄▄▄▄▄▄▄   ║
║  ▄▄▄▄▄▄▄
```

📋 Guide Overview

Section	Topic	Coverage
1	Basics of Shell Scripting	Writing Simple Bash Scripts
1a	Writing Simple Bash Scripts	Variables, Control Flow, Functions
1b	Scheduling Scripts with cron	cron, crontab, at, systemd timers
2	File Searching and Archiving	find, locate, tar, gzip
2a	Using find & locate	Search, filter, execute on results

Target Audience: Complete beginners with no scripting experience
Format: Hands-on, example-driven, visually rich
Goal: Write production-quality shell scripts and manage files like a pro

---

📚 Table of Contents

- Section 1.0 – Introduction to Shell Scripting
- Section 1a – Writing Simple Bash Scripts
  - Your First Script
  - Variables and Data Types
  - User Input
  - Conditional Logic
  - Loops
  - Functions

    - Arrays
    - String Operations
    - Arithmetic
    - Script Arguments
    - Exit Codes and Error Handling
    - Debugging Scripts
- Section 1b – Executing and Scheduling Scripts with cron
    - Making Scripts Executable
    - cron and crontab
    - Special cron Schedules
    - The at Command
    - Logging cron Output
    - systemd Timers
- Section 2.0 – File Searching and Archiving
- Section 2a – Using find, locate, tar, gzip
    - The find Command
    - The locate Command
    - find vs locate
    - The tar Command
    - Compression: gzip, bzip2, xz
    - Practical Archiving Workflows
- Complete Cheat Sheet

Section 1.0

Introduction to Shell Scripting

🎯 Learning Objectives

- Understand what a shell and shell script are
- Know why shell scripting is a fundamental sysadmin/DevOps skill
- Identify the difference between interactive shell use and scripting
- Understand the Bash shell's place in the Linux ecosystem

What is a Shell?

A **shell** is a program that takes commands from the keyboard (or a file) and gives them to the operating system to execute. It's the interface between you and the OS kernel.

```
WHAT HAPPENS WHEN YOU TYPE A COMMAND
═══════════════════════════════════════════════════════════════

You type: ls -la /home

  ┌─────────────────────────────────────────────────┐
  │  YOUR TERMINAL                                  │
  │  You → type "ls -la /home"                      │
  └─────────────────────┬───────────────────────────┘
                        │
                        ▼
  ┌─────────────────────────────────────────────────┐
  │  THE SHELL (Bash)                               │
  │  Parses your command                            │
  │  Finds the 'ls' program                         │
  │  Passes arguments (-la, /home) to it            │
  └─────────────────────┬───────────────────────────┘
                        │
                        ▼
  ┌─────────────────────────────────────────────────┐
  │  LINUX KERNEL                                   │
  │  Executes the ls program                        │
  │  Reads the /home directory                      │
  │  Returns results                                │
  └─────────────────────┬───────────────────────────┘
                        │
                        ▼
  ┌─────────────────────────────────────────────────┐
  │  OUTPUT displayed back in your terminal         │
  └─────────────────────────────────────────────────┘
```

The Shell Family Tree

```
COMMON SHELLS
═══════════════════════════════════════════════════════════

sh   → The original Bourne Shell (1979). Very basic. Not interactive.
bash → Bourne Again SHell (1989). Default on most Linux systems.
       The most important shell to learn. ← WE USE THIS
zsh  → Z Shell. Default on modern macOS. More features than bash.
fish → Friendly Interactive SHell. Very user-friendly. Nonstandard syntax.
dash → Debian Almquist SHell. Minimal, fast. Used for /bin/sh on Ubuntu.
ksh  → Korn Shell. Common in older enterprise UNIX.

CHECK YOUR SHELL:
  echo $SHELL       → Shows your login shell  (e.g., /bin/bash)
  echo $0           → Shows current shell     (e.g., bash)
  bash --version    → Shows bash version
```

What is a Shell Script?

A **shell script** is a text file containing a sequence of shell commands. Instead of typing commands one by one, you put them in a file and run the file. The shell reads it top-to-bottom and executes each line.

```
INTERACTIVE vs. SCRIPTED USE
═══════════════════════════════════════════════════════════════

INTERACTIVE (typing commands manually):
  $ mkdir backups
  $ cp *.log backups/
  $ tar -czf backups.tar.gz backups/
  $ rm -rf backups/
  (4 commands typed manually, every single time)

SCRIPTED (write once, run anytime):
  ┌─ backup.sh ──────────────────────────────┐
  │ #!/bin/bash                              │
  │ mkdir backups                            │
  │ cp *.log backups/                        │
  │ tar -czf backups.tar.gz backups/         │
  │ rm -rf backups/                          │
  │ echo "Backup complete!"                  │
  └──────────────────────────────────────────┘

  Run with: ./backup.sh
  → Executes all 5 steps automatically, every time

REAL-WORLD USE CASES FOR SHELL SCRIPTS:
  ✅ Automated backups (run every night at 2 AM)
  ✅ System monitoring (check disk/CPU/memory)
  ✅ Deployment pipelines (test → build → deploy)
  ✅ Log rotation and cleanup
  ✅ User account management
  ✅ Data processing (parse logs, generate reports)
```

```
  ✅ Software installation and configuration
  ✅ Batch file renaming / processing
```

Why Bash Specifically?

```
WHY LEARN BASH?
═══════════════════════════════════════════════════════════

✅ Pre-installed on virtually every Linux/Unix/macOS system
✅ Default shell for most cloud servers (AWS, GCP, Azure VMs)
✅ Required knowledge for DevOps, SysAdmin, and Cloud roles
✅ Docker, CI/CD pipelines, Kubernetes — all use bash scripts
✅ POSIX-compatible — scripts work across different Unix systems
✅ Fastest path from "idea" to "automated solution"

BASH IS EVERYWHERE:
  Linux servers          → /bin/bash default
  macOS                  → /bin/bash (or zsh, which reads bash scripts)
  Docker containers      → bash scripts for ENTRYPOINT/CMD
  GitHub Actions         → YAML files run bash commands
  Jenkins / GitLab CI    → bash for build steps
  AWS Lambda             → bash for shell-based functions
  Ansible playbooks      → bash for command modules
```

Section 1a

Writing Simple Bash Scripts

🎯 Learning Objectives

- Create, write, and execute a bash script
- Use variables, conditionals, loops, and functions
- Handle user input and script arguments
- Perform arithmetic and string operations
- Write robust scripts with proper error handling

Your First Script

The Anatomy of a Bash Script

```
#!/bin/bash
# ↑ This is the "shebang" — tells the OS which interpreter to use
# Lines starting with # are COMMENTS — ignored by bash

# Script: hello.sh
# Author: Your Name
# Date:   2024-01-01
# Purpose: My first bash script

# A simple command
echo "Hello, World!"

# Another command
echo "Today's date is: $(date)"

# End of script
```

The Shebang Line (`#!/bin/bash`)

```
THE SHEBANG EXPLAINED
═══════════════════════════════════════════════════════════


#!/bin/bash
│└────────┘
│  Path to the interpreter (bash program)
└── #! = "shebang" or "hashbang" — special marker

When you run: ./myscript.sh
The OS reads the first line, sees #!/bin/bash,
then runs: /bin/bash myscript.sh

COMMON SHEBANG VARIANTS:
  #!/bin/bash          → Bash (explicit path, most common)
  #!/usr/bin/env bash  → Bash (searches PATH — more portable)
  #!/bin/sh            → POSIX sh (minimal, most portable)
  #!/usr/bin/env python3 → Python (same concept!)

RECOMMENDATION: Use #!/bin/bash for bash-specific scripts.
                Use #!/bin/sh for maximum portability.
```

Creating and Running Your First Script

```
# Step 1: Create the script file
nano hello.sh
# OR
vim hello.sh
# OR
cat > hello.sh << 'EOF'
#!/bin/bash
# My first script

echo "Hello, World!"
echo "Current user: $USER"
echo "Current directory: $PWD"
echo "Date and time: $(date)"
echo "Hostname: $(hostname)"
EOF

# Step 2: Make it executable
chmod +x hello.sh
# (Without this, bash won't run it as a program)

# Step 3: Run it
./hello.sh

# EXPECTED OUTPUT:
# Hello, World!
# Current user: alice
# Current directory: /home/alice
# Date and time: Mon Jan 15 09:23:41 UTC 2024
# Hostname: myserver
```

```
# Alternative ways to run a script:
bash hello.sh       # Run with bash explicitly (doesn't need chmod +x)
source hello.sh     # Run in CURRENT shell (variables persist after!)
. hello.sh          # Same as source
```

```
CHMOD PERMISSIONS EXPLAINED
═══════════════════════════════════════════════════════════

chmod +x hello.sh   → Add execute permission for everyone

Permission breakdown:
  r = read    (4)
  w = write   (2)
  x = execute (1)

chmod 755 hello.sh  → rwxr-xr-x (owner: rwx, group: rx, others: rx)
chmod 700 hello.sh  → rwx------ (only owner can read/write/execute)
chmod +x  hello.sh  → Adds execute bit to all (quickest way)

CHECK PERMISSIONS:
  ls -l hello.sh
  -rwxr-xr-x 1 alice alice 147 Jan 15 09:23 hello.sh
   ↑↑↑↑↑↑↑↑↑
   |||||||||└── Other: can read, execute
   ||||||└─── Group: can read, execute
   |||└──── Owner: can read, write, execute
   └─────── File type (- = regular file)
```

Hands-on Exercise 1: Hello World Script

```
# Create a "system info" script that displays:
# 1. A greeting with the current user's name
# 2. The current date and time
# 3. The system's uptime
# 4. Available disk space on /
# 5. Current memory usage

cat > sysinfo.sh << 'EOF'
#!/bin/bash
# Script: sysinfo.sh
# Purpose: Display key system information

echo "═══════════════════════════════"
echo "  SYSTEM INFORMATION REPORT"
echo "═══════════════════════════════"
echo ""
echo "👤 User:        $USER"
echo "📅 Date/Time:   $(date)"
echo "⏱  Uptime:      $(uptime -p)"
echo ""
echo "💾 Disk Usage (/):"
df -h / | tail -1
echo ""
echo "🧠 Memory Usage:"
free -h | grep Mem
echo ""
echo "═══════════════════════════════"
EOF
```

```
chmod +x sysinfo.sh
./sysinfo.sh
```

Variables and Data Types

Declaring and Using Variables

```bash
#!/bin/bash

# DECLARING VARIABLES
# No spaces around = (this is critical!)
name="Alice"          # String variable
age=30                # Integer (stored as string internally)
pi=3.14159            # Float (bash treats as string; use bc for math)
is_admin=true         # Boolean (convention — bash has no true bool type)

# WRONG — spaces cause errors:
# name = "Alice"      ← ERROR: bash thinks 'name' is a command
# age =30             ← ERROR
# age= 30             ← ERROR

# USING VARIABLES (prefix with $)
echo "Name: $name"
echo "Age: $age"

# CURLY BRACES (for clarity and complex usage)
echo "Hello, ${name}!"          # Same as $name, but explicit
echo "The file is ${name}.txt"  # Without {}, bash reads $name.txt literally
                                # which would look for variable "nametxt"

# VARIABLE NAMING RULES:
# ✅ name   my_var   FILE_PATH   var2   _private
# ❌ 2name  my-var   my var      $name (can't start with digit or have spaces/-)
```

```
# CONVENTIONS (not enforced by bash):
MY_CONSTANT="fixed"   # UPPERCASE = constants or environment variables
local_var="dynamic"   # lowercase = local script variables
```

Variable Types and Scope

```bash
#!/bin/bash

# === ENVIRONMENT VARIABLES (inherited from shell) ===
echo "Home directory: $HOME"
echo "Current user: $USER"
echo "Shell: $SHELL"
echo "PATH: $PATH"
echo "Language: $LANG"

# === READONLY VARIABLES ===
readonly MAX_RETRIES=3
# MAX_RETRIES=5  ← Would cause error: "readonly variable"

# === UNSET VARIABLES ===
echo "Undefined: ${UNDEFINED_VAR}"      # Prints nothing (empty)
echo "With default: ${UNDEFINED_VAR:-default_value}"  # Prints: default_value

# VARIABLE EXPANSION TRICKS:
filename="report.tar.gz"
echo "${filename}"                  # report.tar.gz
echo "${#filename}"                 # 13 (length of string)
echo "${filename%.*}"               # report.tar (remove shortest suffix)
echo "${filename%%.*}"              # report (remove longest suffix)
echo "${filename#*.}"               # tar.gz (remove shortest prefix)
echo "${filename##*.}"              # gz (remove longest prefix — get extension)
echo "${filename/report/summary}"   # summary.tar.gz (replace first)
echo "${filename//a/A}"             # report.tAr.gz (replace all)

# === LOCAL SCOPE IN FUNCTIONS ===
```

```
my_var="global"

my_function() {
    local my_var="local"   # 'local' restricts to this function
    echo "Inside function: $my_var"    # Prints: local
}

my_function
echo "Outside function: $my_var"       # Prints: global
```

Special Built-in Variables

```
#!/bin/bash
# Special variables bash sets automatically

echo "Script name:     $0"    # Name of the script
echo "First arg:       $1"    # First command-line argument
echo "Second arg:      $2"    # Second argument
echo "All arguments:   $@"    # All arguments as separate strings
echo "All arguments:   $*"    # All arguments as one string
echo "Argument count:  $#"    # Number of arguments
echo "Process ID:      $$"    # PID of current script
echo "Last PID:        $!"    # PID of last background command
echo "Last exit code:  $?"    # Exit status of last command (0=success)
```

```
SPECIAL VARIABLES QUICK REFERENCE:
═══════════════════════════════════════════════════════════

$0    Script name
$1-$9 Positional parameters (arguments)
${10} Arguments beyond 9 need curly braces
$#    Number of arguments
$@    All arguments (preserves individual quoting — PREFER THIS)
$*    All arguments (single string when quoted)
$$    Current process ID
$!    PID of last background process
$?    Exit code of last command (0=success, non-zero=error)
$-    Current shell option flags
```

User Input

```bash
#!/bin/bash

# === BASIC INPUT ===
echo "What is your name?"
read name
echo "Hello, $name!"

# Read with a prompt on same line:
read -p "Enter your age: " age
echo "You are $age years old."

# Silent input (for passwords):
read -sp "Enter password: " password
echo ""   # Newline after silent input
echo "Password received (length: ${#password})"

# Read with a timeout (5 seconds):
read -t 5 -p "Enter something (5 sec timeout): " input
if [ $? -ne 0 ]; then
    echo "Timed out! Using default."
    input="default"
fi

# Read multiple values at once:
read -p "Enter first and last name: " first_name last_name
echo "First: $first_name, Last: $last_name"

# Read into an array:
```

```bash
read -p "Enter three colors (space-separated): " -a colors
echo "First color: ${colors[0]}"
echo "All colors: ${colors[@]}"

# Read line by line from a file:
while IFS= read -r line; do
    echo "Line: $line"
done < /etc/hostname
```

Input Validation

```bash
#!/bin/bash
# validate_input.sh — Demonstrates robust input validation

get_valid_age() {
    while true; do
        read -p "Enter your age (1-120): " age

        # Check if it's a number
        if ! [[ "$age" =~ ^[0-9]+$ ]]; then
            echo "❌ Error: Please enter a valid number."
            continue
        fi

        # Check if it's in range
        if [ "$age" -lt 1 ] || [ "$age" -gt 120 ]; then
            echo "❌ Error: Age must be between 1 and 120."
            continue
        fi

        # Valid input — exit loop
        break
    done
    echo "✅ Valid age entered: $age"
}

get_valid_age
```

Conditional Logic

The `if` Statement

```bash
#!/bin/bash

# BASIC IF SYNTAX:
if [ condition ]; then
    # commands
elif [ condition ]; then
    # commands
else
    # commands
fi

# REAL EXAMPLE:
age=25

if [ $age -lt 18 ]; then
    echo "You are a minor."
elif [ $age -lt 65 ]; then
    echo "You are an adult."
else
    echo "You are a senior."
fi
```

Test Conditions — The Complete Guide

```bash
#!/bin/bash

# ═══════════════════════════════════════════════════════════════
# NUMERIC COMPARISONS
# ═══════════════════════════════════════════════════════════════
# -eq   Equal to               [ $a -eq $b ]
# -ne   Not equal to           [ $a -ne $b ]
# -lt   Less than              [ $a -lt $b ]
# -le   Less than or equal     [ $a -le $b ]
# -gt   Greater than           [ $a -gt $b ]
# -ge   Greater than or equal  [ $a -ge $b ]

num=42
if [ $num -ge 10 ] && [ $num -le 100 ]; then
    echo "$num is between 10 and 100"
fi

# ═══════════════════════════════════════════════════════════════
# STRING COMPARISONS
# ═══════════════════════════════════════════════════════════════
# =    or ==  Equal          [ "$a" = "$b" ]  or [[ $a == $b ]]
# !=         Not equal       [ "$a" != "$b" ]
# <          Less than       [[ "$a" < "$b" ]]  (lexicographic)
# >          Greater than    [[ "$a" > "$b" ]]
# -z         Empty string    [ -z "$a" ]
# -n         Non-empty       [ -n "$a" ]

name="Alice"
if [ -n "$name" ]; then
```

```
    echo "Name is not empty: $name"
fi

if [[ "$name" == A* ]]; then   # Pattern matching (only with [[ ]])
    echo "Name starts with A"
fi

# ALWAYS quote string variables in [ ] tests!
# Unquoted: [ $name = "Alice" ]  — breaks if $name is empty or has spaces
# Quoted:   [ "$name" = "Alice" ] — always safe

# ═══════════════════════════════════════════════════════════════
# FILE AND DIRECTORY TESTS
# ═══════════════════════════════════════════════════════════════
# -e   Exists                [ -e "$path" ]
# -f   Is a regular file     [ -f "$file" ]
# -d   Is a directory        [ -d "$dir" ]
# -r   Is readable           [ -r "$file" ]
# -w   Is writable           [ -w "$file" ]
# -x   Is executable         [ -x "$file" ]
# -s   Is non-empty (size>0) [ -s "$file" ]
# -L   Is a symbolic link    [ -L "$path" ]
# -nt  Newer than            [ "$file1" -nt "$file2" ]
# -ot  Older than            [ "$file1" -ot "$file2" ]

filepath="/etc/hosts"
if [ -f "$filepath" ]; then
    echo "File exists and is a regular file"
fi

if [ -r "$filepath" ] && [ -s "$filepath" ]; then
    echo "File is readable and non-empty"
```

```
fi

# ═══════════════════════════════════════════════════════════════
# COMBINING CONDITIONS
# ═══════════════════════════════════════════════════════════════
# && (AND)    [ cond1 ] && [ cond2 ]   OR   [[ cond1 && cond2 ]]
# || (OR)     [ cond1 ] || [ cond2 ]   OR   [[ cond1 || cond2 ]]
# !  (NOT)    [ ! condition ]

x=15
if [ $x -gt 10 ] && [ $x -lt 20 ]; then
    echo "$x is between 10 and 20"
fi

if [ ! -d "/tmp/mydir" ]; then
    echo "Directory does not exist, creating..."
    mkdir /tmp/mydir
fi
```

`[ ]` vs `[[ ]]` vs `(( ))`

```
THREE TYPES OF TEST IN BASH
═══════════════════════════════════════════════════════════

[ ]   — POSIX test. Works in sh and bash.
        Must quote variables. No pattern matching. No &&/||.

[[ ]] — Bash extended test. BASH ONLY (not sh).
        No need to quote simple variables.
        Supports pattern matching (==, =~).
        Supports && and || inside.

(( )) — Arithmetic evaluation. Bash only.
        For integer math and comparison. No $ needed on variables.

EXAMPLES:
  POSIX:  [ "$x" -gt 10 ]           ← Numeric comparison
  Bash:   [[ "$name" == A* ]]        ← Pattern match
  Bash:   [[ "$str" =~ ^[0-9]+$ ]]   ← Regex match
  Arith:  (( x > 10 ))              ← No $ needed!
  Arith:  (( x++ ))                 ← Increment x
  Arith:  (( result = x * y ))      ← Multiply

RECOMMENDATION:
  Use [[ ]] for all string/file tests in bash scripts
  Use (( )) for all arithmetic tests
  Use [ ] only when writing POSIX-portable sh scripts
```

The `case` Statement

```bash
#!/bin/bash
# case is cleaner than long if-elif chains for multiple options

read -p "Enter a day of the week: " day

case "$day" in
    Monday|Mon)
        echo "Start of the work week. Coffee time! ☕"
        ;;
    Tuesday|Tue|Wednesday|Wed|Thursday|Thu)
        echo "Midweek grind. You've got this! 💪"
        ;;
    Friday|Fri)
        echo "TGIF! Weekend is near! 🎉"
        ;;
    Saturday|Sun|Sunday)
        echo "It's the weekend! Relax. 🌴"
        ;;
    *)
        echo "Unknown day: $day"
        exit 1
        ;;
esac

# CASE PATTERNS:
#   word)     Exact match
#   Mon|Tue)  OR — matches either
#   Mon*)     Wildcard — starts with Mon
```

```
#   [Mm]on)   Character class — Mon or mon
#   *)        Default — matches anything (like else)
```

Practical Example: System Health Check

```bash
#!/bin/bash
# health_check.sh — Checks system health with conditionals

DISK_THRESHOLD=80   # Alert if disk usage exceeds 80%
MEM_THRESHOLD=90    # Alert if memory usage exceeds 90%

echo "═══════════════════════════════"
echo "  SYSTEM HEALTH CHECK"
echo "═══════════════════════════════"

# --- DISK CHECK ---
disk_usage=$(df / | tail -1 | awk '{print $5}' | tr -d '%')

if [ "$disk_usage" -ge "$DISK_THRESHOLD" ]; then
    echo "⚠️  DISK: ${disk_usage}% used — CRITICAL!"
elif [ "$disk_usage" -ge 60 ]; then
    echo "⚡ DISK: ${disk_usage}% used — WARNING"
else
    echo "✅ DISK: ${disk_usage}% used — OK"
fi

# --- MEMORY CHECK ---
mem_total=$(free | grep Mem | awk '{print $2}')
mem_used=$(free | grep Mem | awk '{print $3}')
mem_pct=$(( mem_used * 100 / mem_total ))

if [ "$mem_pct" -ge "$MEM_THRESHOLD" ]; then
    echo "⚠️  MEM:  ${mem_pct}% used — CRITICAL!"
elif [ "$mem_pct" -ge 70 ]; then
```

```
    echo "⚡ MEM:  ${mem_pct}% used — WARNING"
else
    echo "✅ MEM:  ${mem_pct}% used — OK"
fi

# --- IMPORTANT SERVICES ---
for service in ssh cron; do
    if systemctl is-active --quiet "$service" 2>/dev/null; then
        echo "✅ SERVICE: $service is running"
    else
        echo "❌ SERVICE: $service is NOT running"
    fi
done

echo "═══════════════════════════════"
```

Loops

The `for` Loop

```bash
#!/bin/bash

# FOR loop - iterate over a list
for item in apple banana cherry; do
    echo "Fruit: $item"
done

# FOR loop - iterate over a range
for i in {1..5}; do
    echo "Number: $i"
done

# FOR loop - range with step
for i in {0..20..5}; do   # 0, 5, 10, 15, 20
    echo "$i"
done

# C-style FOR loop
for (( i=0; i<5; i++ )); do
    echo "Index: $i"
done

# FOR loop - iterate over files
for file in /var/log/*.log; do
    echo "Log file: $file ($(wc -l < "$file") lines)"
done
```

```
# FOR loop — iterate over command output
for user in $(cut -d: -f1 /etc/passwd | head -5); do
    echo "User: $user"
done

# FOR loop — iterate over array
fruits=("apple" "banana" "cherry" "date")
for fruit in "${fruits[@]}"; do
    echo "Fruit: $fruit"
done
```

The `while` Loop

```bash
#!/bin/bash

# WHILE loop — runs while condition is TRUE
count=1
while [ $count -le 5 ]; do
    echo "Count: $count"
    (( count++ ))   # Increment counter
done

# WHILE loop — reading a file line by line (BEST PRACTICE)
while IFS= read -r line; do
    echo "Processing: $line"
done < "/etc/hosts"

# WHILE loop — reading command output
while IFS= read -r line; do
    echo "Process: $line"
done < <(ps aux | head -5)   # Process substitution

# INFINITE LOOP with break
attempt=0
max_attempts=3
while true; do
    (( attempt++ ))
    read -p "Enter 'quit' to exit (attempt $attempt/$max_attempts): " input

    if [[ "$input" == "quit" ]]; then
        echo "Exiting."
        break
```

```
    fi

    if [ $attempt -ge $max_attempts ]; then
        echo "Max attempts reached."
        break
    fi
done

# WHILE with continue (skip to next iteration)
count=0
while [ $count -lt 10 ]; do
    (( count++ ))
    if (( count % 2 == 0 )); then
        continue   # Skip even numbers
    fi
    echo "Odd: $count"
done
```

The `until` Loop

```
#!/bin/bash

# UNTIL loop — runs until condition becomes TRUE (opposite of while)
count=1
until [ $count -gt 5 ]; do
    echo "Count: $count"
    (( count++ ))
done

# Practical: Wait until a file exists
until [ -f "/tmp/ready.flag" ]; do
    echo "Waiting for ready.flag..."
    sleep 5
done
echo "Flag found! Proceeding."
```

Loop Control: `break` and `continue`

```bash
#!/bin/bash

# BREAK — exit the loop entirely
echo "Break example:"
for i in {1..10}; do
    if [ $i -eq 5 ]; then
        echo "Breaking at $i"
        break
    fi
    echo $i
done
# Output: 1 2 3 4 Breaking at 5

# CONTINUE — skip to next iteration
echo "Continue example:"
for i in {1..10}; do
    if (( i % 3 == 0 )); then
        continue   # Skip multiples of 3
    fi
    echo -n "$i "
done
echo ""
# Output: 1 2 4 5 7 8 10
```

Real-World Loop Example: Batch File Processing

```bash
#!/bin/bash
# batch_rename.sh — Rename all .txt files to .bak in a directory

SOURCE_DIR="${1:-.}"   # Use first argument or current directory
RENAMED=0
FAILED=0

echo "Processing files in: $SOURCE_DIR"
echo "──────────────────────────────────"

for file in "$SOURCE_DIR"/*.txt; do
    # Check if glob matched anything (no .txt files = loop runs once with literal)
    if [ ! -f "$file" ]; then
        echo "No .txt files found."
        break
    fi

    # Build new filename
    new_name="${file%.txt}.bak"

    # Attempt rename
    if mv "$file" "$new_name" 2>/dev/null; then
        echo "✅ Renamed: $(basename "$file") → $(basename "$new_name")"
        (( RENAMED++ ))
    else
        echo "❌ Failed:  $(basename "$file")"
        (( FAILED++ ))
    fi
done
```

```
echo "──────────────────────────────────"
echo "Renamed: $RENAMED | Failed: $FAILED"
```

Functions

Defining and Calling Functions

```bash
#!/bin/bash

# FUNCTION DEFINITION SYNTAX (two equivalent forms):

# Form 1: function keyword
function greet() {
    echo "Hello, $1!"
}

# Form 2: No keyword (POSIX-compatible, preferred)
greet() {
    echo "Hello, $1!"
}

# CALLING A FUNCTION:
greet "World"       # Output: Hello, World!
greet "Alice"       # Output: Hello, Alice!

# FUNCTIONS WITH MULTIPLE PARAMETERS:
create_user() {
    local username="$1"
    local home_dir="$2"
    local shell="${3:-/bin/bash}"   # Default value if not provided

    echo "Creating user: $username"
    echo "  Home: $home_dir"
```

```
    echo "  Shell: $shell"
    # useradd -m -d "$home_dir" -s "$shell" "$username"  # Actual command
}

create_user "alice" "/home/alice"
create_user "bob"   "/home/bob"   "/bin/zsh"
```

Return Values and Exit Codes

```bash
#!/bin/bash

# BASH FUNCTIONS DON'T "return" values like most languages.
# They return EXIT CODES (0-255) via 'return'.
# To return actual data: echo the value and capture with $()

# Method 1: Return exit codes (success/failure)
is_even() {
    local num="$1"
    if (( num % 2 == 0 )); then
        return 0   # 0 = success = true
    else
        return 1   # Non-zero = failure = false
    fi
}

if is_even 42; then
    echo "42 is even"
fi

if ! is_even 7; then
    echo "7 is not even"
fi

# Method 2: Echo data, capture with $()
get_username() {
    local uid="$1"
    echo "$(getent passwd "$uid" | cut -d: -f1)"
}
```

```
username=$(get_username 1000)
echo "Username for UID 1000: $username"

# Method 3: Use a global variable (common but less clean)
calculate_area() {
    RESULT=$(( $1 * $2 ))   # Set global variable
}

calculate_area 5 10
echo "Area: $RESULT"
```

Function Best Practices

```bash
#!/bin/bash
# functions_best_practices.sh

# ─── USE LOCAL VARIABLES ──────────────────────────────────────
bad_function() {
    result="calculated value"   # Global! Pollutes namespace
}

good_function() {
    local result="calculated value"   # Local — stays in function
    echo "$result"                    # Output it instead
}

# ─── VALIDATE INPUTS ──────────────────────────────────────────
divide() {
    local dividend="$1"
    local divisor="$2"

    # Validate arguments exist
    if [ -z "$dividend" ] || [ -z "$divisor" ]; then
        echo "Error: divide requires two arguments" >&2
        return 1
    fi

    # Validate not dividing by zero
    if [ "$divisor" -eq 0 ]; then
        echo "Error: Division by zero" >&2
        return 1
    fi
```

```
    echo $(( dividend / divisor ))
}

result=$(divide 100 4)
echo "100 / 4 = $result"

divide 100 0   # Will print error to stderr and return 1

# ─── DOCUMENT YOUR FUNCTIONS ──────────────────────────────
# Function: backup_file
# Description: Creates a timestamped backup of a file
# Arguments:
#   $1 - Path to file to backup
#   $2 - (optional) Backup directory (default: same as source)
# Returns:
#   0 - Success
#   1 - File not found
#   2 - Backup failed
backup_file() {
    local source_file="$1"
    local backup_dir="${2:-$(dirname "$source_file")}"
    local timestamp
    timestamp=$(date +%Y%m%d_%H%M%S)
    local backup_name
    backup_name="${backup_dir}/$(basename "$source_file").${timestamp}.bak"

    if [ ! -f "$source_file" ]; then
        echo "Error: File not found: $source_file" >&2
        return 1
    fi
```

```
    if cp "$source_file" "$backup_name"; then
        echo "✅ Backup created: $backup_name"
        return 0
    else
        echo "❌ Backup failed for: $source_file" >&2
        return 2
    fi
}
```

Library Pattern — Reusable Function Files

```bash
#!/bin/bash
# lib/utils.sh — A reusable library of utility functions

# ─── LOGGING FUNCTIONS ──────────────────────────────────────
LOG_FILE="/var/log/myscript.log"

log_info() {
    local message="$1"
    local timestamp
    timestamp=$(date '+%Y-%m-%d %H:%M:%S')
    echo "[INFO]  $timestamp — $message" | tee -a "$LOG_FILE"
}

log_warn() {
    local message="$1"
    local timestamp
    timestamp=$(date '+%Y-%m-%d %H:%M:%S')
    echo "[WARN]  $timestamp — $message" | tee -a "$LOG_FILE" >&2
}

log_error() {
    local message="$1"
    local timestamp
    timestamp=$(date '+%Y-%m-%d %H:%M:%S')
    echo "[ERROR] $timestamp — $message" | tee -a "$LOG_FILE" >&2
}

# ─── UTILITY FUNCTIONS ──────────────────────────────────────
is_root() {
```

```
    [ "$(id -u)" -eq 0 ]
}

file_exists() {
    [ -f "$1" ]
}

dir_exists() {
    [ -d "$1" ]
}

command_exists() {
    command -v "$1" &>/dev/null
}
```

```bash
#!/bin/bash
# main_script.sh — Sources the library

# Source the utility library
source ./lib/utils.sh   # OR: . ./lib/utils.sh

# Now all library functions are available:
log_info "Script started"

if ! is_root; then
    log_error "This script must be run as root"
    exit 1
fi

if command_exists docker; then
    log_info "Docker is installed"
else
    log_warn "Docker not found — some features disabled"
fi
```

Arrays

```bash
#!/bin/bash

# ─── INDEXED ARRAYS ──────────────────────────────────────────
# Declaration:
fruits=("apple" "banana" "cherry" "date")
# OR element by element:
colors[0]="red"
colors[1]="green"
colors[2]="blue"

# Access elements:
echo "${fruits[0]}"        # apple (first element)
echo "${fruits[2]}"        # cherry (third element)
echo "${fruits[-1]}"       # date (last element)
echo "${fruits[@]}"        # All elements
echo "${#fruits[@]}"       # 4 (number of elements)
echo "${!fruits[@]}"       # 0 1 2 3 (indices)

# Modify:
fruits[4]="elderberry"     # Add fifth element
fruits+=("fig")            # Append
unset fruits[1]            # Delete element (leaves gap)

# Slice:
echo "${fruits[@]:1:2}"    # Elements 1 and 2 (banana cherry)

# Iterate:
for fruit in "${fruits[@]}"; do
```

```bash
    echo "  → $fruit"
done

# ── ASSOCIATIVE ARRAYS (key-value maps) ──────────────────
# Requires Bash 4+
declare -A person
person[name]="Alice"
person[age]="30"
person[city]="London"

echo "${person[name]}"          # Alice
echo "${person[@]}"             # All values
echo "${!person[@]}"            # All keys

for key in "${!person[@]}"; do
    echo "$key = ${person[$key]}"
done

# ── PRACTICAL: MENU SYSTEM ───────────────────────────────
options=("Install software" "Update system" "Check status" "Quit")

echo "Please select an option:"
select opt in "${options[@]}"; do
    case $opt in
        "Install software") echo "Installing..." ;;
        "Update system")    echo "Updating..."   ;;
        "Check status")     echo "Checking..."   ;;
        "Quit")             break                ;;
        *) echo "Invalid option: $REPLY"         ;;
    esac
done
```

String Operations

```bash
#!/bin/bash

str="Hello, World! Welcome to Bash."

# ── LENGTH ──────────────────────────────────────────
echo "${#str}"           # 30

# ── SUBSTRING EXTRACTION ────────────────────────────
echo "${str:0:5}"        # Hello (from pos 0, length 5)
echo "${str:7}"          # World! Welcome to Bash. (from pos 7)
echo "${str: -5}"        # Bash. (last 5 chars — space before -)

# ── CASE CONVERSION (Bash 4+) ───────────────────────
echo "${str,,}"          # hello, world! welcome to bash. (all lowercase)
echo "${str^^}"          # HELLO, WORLD! WELCOME TO BASH. (all uppercase)
echo "${str^}"           # Hello, World! ... (capitalize first char)

# ── SUBSTITUTION ────────────────────────────────────
echo "${str/World/Bash}" # Hello, Bash! Welcome to Bash. (first only)
echo "${str//o/0}"       # Hell0, W0rld! Welc0me t0 Bash. (all)

# ── TRIMMING ────────────────────────────────────────
filename="report.tar.gz"
echo "${filename%.gz}"   # report.tar (remove .gz suffix)
echo "${filename%.*}"    # report.tar (remove last extension)
echo "${filename%%.*}"   # report (remove all extensions)
path="/usr/local/bin/script.sh"
echo "${path##*/}"       # script.sh (get filename — remove all before /)
```

```
echo "${path%/*}"        # /usr/local/bin (get dir — remove filename)

# ── DEFAULT VALUES ──────────────────────────────────────
unset myvar
echo "${myvar:-default}"      # "default" (use default if unset/empty)
echo "${myvar:=default}"      # "default" AND sets $myvar = "default"
echo "${myvar:?Error msg}"    # Prints error and exits if unset/empty
echo "${myvar:+other}"        # "other" if myvar IS set (opposite)

# ── CHECKING STRINGS ────────────────────────────────────
check_string() {
    local str="$1"
    [[ -z "$str" ]]  && echo "Empty"   && return
    [[ -n "$str" ]]  && echo "Non-empty: $str"
    [[ "$str" =~ ^[0-9]+$ ]] && echo "Is a number"
    [[ "$str" =~ ^[a-zA-Z]+$ ]] && echo "Is alphabetic only"
}

check_string ""
check_string "hello"
check_string "12345"
```

Arithmetic

```bash
#!/bin/bash

# ─── INTEGER ARITHMETIC ──────────────────────────────────
# Method 1: $(( )) — arithmetic expansion (preferred)
a=10
b=3
echo $(( a + b ))   # 13
echo $(( a - b ))   # 7
echo $(( a * b ))   # 30
echo $(( a / b ))   # 3 (integer division — truncates!)
echo $(( a % b ))   # 1 (modulo / remainder)
echo $(( a ** b ))  # 1000 (exponentiation — Bash 4+)

# Increment / Decrement:
(( a++ ))    # Post-increment: a = a + 1
(( a-- ))    # Post-decrement: a = a - 1
(( a += 5 )) # a = a + 5
(( a *= 2 )) # a = a * 2

# Method 2: expr (old way — avoid unless needed for POSIX sh)
result=$(expr 10 + 3)    # 13 (spaces required!)
result=$(expr 10 \* 3)   # 30 (must escape *)

# Method 3: let command
let "result = 10 + 3"
let "result += 5"

# ─── FLOATING POINT (bc calculator) ──────────────────────
```

```
# Bash doesn't natively support floats — use bc:
result=$(echo "scale=2; 10 / 3" | bc)
echo "10 / 3 = $result"           # 3.33

result=$(echo "scale=4; sqrt(2)" | bc -l)
echo "√2 = $result"               # 1.4142

circumference=$(echo "scale=2; 2 * 3.14159 * 5" | bc)
echo "Circumference (r=5): $circumference"   # 31.41

# ─── PRACTICAL: CALCULATE DISK PERCENTAGE ──────────────────
total_space=$(df / | tail -1 | awk '{print $2}')
used_space=$(df / | tail -1 | awk '{print $3}')
percent_used=$(( used_space * 100 / total_space ))
echo "Disk used: ${percent_used}%"
```

Script Arguments

```bash
#!/bin/bash
# args_demo.sh — Demonstrates argument handling

# ─── BASIC ARGUMENT ACCESS ──────────────────────────────
echo "Script:     $0"       # ./args_demo.sh
echo "Arg 1:      $1"
echo "Arg 2:      $2"
echo "All args:   $@"
echo "Arg count:  $#"

# ─── SHIFT COMMAND ──────────────────────────────────────
# shift moves all arguments left by 1 ($2→$1, $3→$2, etc.)
while [ $# -gt 0 ]; do
    echo "Processing: $1"
    shift
done

# ─── NAMED FLAGS (getopts) ──────────────────────────────
#!/bin/bash
# Usage: ./script.sh -u alice -p /home/alice -v

usage() {
    echo "Usage: $0 -u USERNAME -p PATH [-v] [-h]"
    echo "  -u  Username (required)"
    echo "  -p  Path (required)"
    echo "  -v  Verbose mode"
    echo "  -h  Show this help"
    exit 1
```

```
}

VERBOSE=false
USERNAME=""
USER_PATH=""

while getopts "u:p:vh" opt; do
    case $opt in
        u) USERNAME="$OPTARG" ;;     # -u takes an argument (:)
        p) USER_PATH="$OPTARG" ;;    # -p takes an argument (:)
        v) VERBOSE=true ;;           # -v is a flag (no argument)
        h) usage ;;                  # -h shows help
        ?) usage ;;                  # Unknown option
    esac
done

# Validate required arguments
if [ -z "$USERNAME" ] || [ -z "$USER_PATH" ]; then
    echo "Error: -u and -p are required."
    usage
fi

echo "Username: $USERNAME"
echo "Path:     $USER_PATH"
$VERBOSE && echo "Verbose mode enabled"
```

```
ARGUMENT HANDLING BEST PRACTICES:
═══════════════════════════════════════════════════════════════

ALWAYS validate required arguments:
  if [ -z "$1" ]; then echo "Usage: $0 <argument>"; exit 1; fi

ALWAYS provide usage information:
  usage() { echo "Usage: $0 [-flags] <args>"; exit 1; }

ALWAYS use getopts for named flags
  Supports: -v, -u alice, -p /path

USE long options with getopt (not getopts) for --verbose style
  getopt --options u:p:vh --longoptions user:,path:,verbose,help

DEFAULT VALUES with parameter expansion:
  output_dir="${1:-/tmp/output}"  ← Default if $1 not provided
```

Exit Codes and Error Handling

```bash
#!/bin/bash

# ─── EXIT CODES ──────────────────────────────────────────
# Every command exits with a code: 0 = success, 1-255 = error
ls /tmp     # exits 0 (success)
ls /nope    # exits 2 (file not found)
echo $?     # Print last exit code

# In your scripts:
exit 0      # Script succeeded
exit 1      # General error
exit 2      # Misuse of shell command
exit 127    # Command not found

# ─── SET SAFETY OPTIONS ──────────────────────────────────
#!/bin/bash
set -e          # Exit immediately if any command fails
set -u          # Treat unset variables as errors
set -o pipefail # Pipe fails if any command in it fails
# Shorthand: set -euo pipefail

# ─── ERROR HANDLING PATTERNS ─────────────────────────────

# Pattern 1: Check exit code manually
cp source.txt dest.txt
if [ $? -ne 0 ]; then
    echo "Error: Copy failed" >&2
    exit 1
```

```bash
fi

# Pattern 2: Short-circuit with ||
cp source.txt dest.txt || { echo "Copy failed!" >&2; exit 1; }

# Pattern 3: AND chain
mkdir -p /tmp/output && cp file.txt /tmp/output/ && echo "Done"

# Pattern 4: trap — catch errors and signals
cleanup() {
    echo "Cleaning up temporary files..."
    rm -rf /tmp/script_$$
}

trap cleanup EXIT           # Run cleanup() when script exits (any reason)
trap cleanup INT TERM       # Run on Ctrl+C or kill signal

# Pattern 5: Error function
error_exit() {
    local message="${1:-An unknown error occurred}"
    local code="${2:-1}"
    echo "ERROR: $message" >&2
    exit "$code"
}

# Usage:
[ -f "config.cfg" ] || error_exit "config.cfg not found" 2

# ─── FULL ROBUST SCRIPT TEMPLATE ──────────────────────────────
#!/bin/bash
set -euo pipefail
```

```bash
# Script metadata
readonly SCRIPT_NAME="$(basename "$0")"
readonly SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"
readonly TIMESTAMP=$(date +%Y%m%d_%H%M%S)
readonly LOG_FILE="/tmp/${SCRIPT_NAME%.*}_${TIMESTAMP}.log"

# Logging
log() { echo "[$(date '+%H:%M:%S')] $*" | tee -a "$LOG_FILE"; }
error() { echo "[ERROR] $*" >&2 | tee -a "$LOG_FILE"; }

# Cleanup on exit
cleanup() {
    local exit_code=$?
    log "Script exiting with code: $exit_code"
    # rm -rf "$TEMP_DIR"  # Clean up temp files
}
trap cleanup EXIT

# Validate environment
check_requirements() {
    local missing=()
    for cmd in curl jq python3; do
        command -v "$cmd" &>/dev/null || missing+=("$cmd")
    done

    if [ ${#missing[@]} -gt 0 ]; then
        error "Missing required commands: ${missing[*]}"
        exit 1
    fi
}

check_requirements
```

```
log "Script started: $SCRIPT_NAME"
log "All requirements met. Proceeding..."
```

Debugging Scripts

```bash
#!/bin/bash

# ─── BASH DEBUGGING OPTIONS ──────────────────────────────
set -x    # XTRACE: Print each command before executing
set -v    # VERBOSE: Print each line as it's read (before expansion)
set -n    # Check syntax only, don't execute
set -e    # Exit on error

# Turn on/off for specific sections:
set -x
some_complex_function
set +x    # Turn off xtrace

# ─── RUN SCRIPT WITH DEBUG ───────────────────────────────
bash -x script.sh          # Debug mode for entire script
bash -n script.sh          # Syntax check only (no execution)
bash -v script.sh          # Verbose mode

# ─── DEBUGGING TECHNIQUES ────────────────────────────────

# 1. Print variable values:
echo "DEBUG: variable=$variable"
echo "DEBUG: array=${array[*]}"
declare -p variable         # Print variable with type info

# 2. Print line numbers:
echo "DEBUG: At line $LINENO"
```

```bash
# 3. Use PS4 for better xtrace output:
PS4='+(${BASH_SOURCE}:${LINENO}): ${FUNCNAME[0]:+${FUNCNAME[0]}(): }'
set -x

# 4. Pause for inspection:
read -p "DEBUG: Press Enter to continue at line $LINENO..." _

# 5. Function call tracing:
trap 'echo "Calling: $FUNCNAME"' DEBUG

# ─── COMMON BUG PATTERNS ──────────────────────────────────
# Bug: Forgetting quotes → word splitting
file="my file.txt"
cat $file     # WRONG — bash splits on space → tries to cat "my" and "file.txt"
cat "$file"   # CORRECT — treats as one argument

# Bug: Using = instead of -eq for numbers
if [ "$count" = 5 ]; then    # WRONG — string comparison!
if [ "$count" -eq 5 ]; then  # CORRECT — numeric comparison

# Bug: Forgetting $ on variable
name="Alice"
echo "Hello name"    # WRONG — prints: Hello name
echo "Hello $name"   # CORRECT — prints: Hello Alice

# Bug: Single quotes prevent expansion
echo 'Hello $name'   # Single quotes: literal, prints: Hello $name
echo "Hello $name"   # Double quotes: expands, prints: Hello Alice
```

Section 1b

Executing and Scheduling Scripts with cron

🎯 Learning Objectives

- Make scripts executable and run them correctly
- Understand cron and crontab syntax
- Schedule scripts to run at specific times
- Use `at` for one-time scheduled tasks
- Log and monitor scheduled job output
- Understand systemd timers as a modern alternative

Making Scripts Executable

```
# ─── MAKE EXECUTABLE ──────────────────────────────────────
chmod +x script.sh      # Add execute permission
chmod 755 script.sh     # rwxr-xr-x (owner: all, others: read+execute)
chmod 700 script.sh     # rwx------ (private — only owner can execute)

# ─── RUNNING SCRIPTS ──────────────────────────────────────

# Method 1: With explicit path (most common for scheduled scripts)
./myscript.sh           # From current directory
/home/alice/scripts/myscript.sh    # Full absolute path

# Method 2: With bash explicitly (no chmod +x needed)
bash myscript.sh
bash /home/alice/scripts/myscript.sh

# Method 3: In PATH (so you can run it like a built-in command)
sudo cp myscript.sh /usr/local/bin/myscript
sudo chmod +x /usr/local/bin/myscript
myscript     # Now runs from anywhere!

# Method 4: Source (runs in current shell — variables persist)
source myscript.sh
. myscript.sh           # Equivalent shorthand

# ─── SHEBANG PATH MATTERS ─────────────────────────────────
# For cron jobs, always use ABSOLUTE paths in shebang AND commands:
#!/bin/bash              # Good — absolute path to bash
#!/usr/bin/env bash      # Better — finds bash in PATH
```

```
# Inside cron scripts, $PATH may be minimal, so use full paths:
/bin/cp source dest      # Not just: cp source dest
/usr/bin/python3 script.py
/usr/local/bin/node app.js
```

cron and crontab

What is cron?

```
CRON OVERVIEW
═══════════════════════════════════════════════════════════

cron   → A background daemon (service) that runs scheduled jobs.
         It wakes up every minute, checks crontab files,
         and runs any jobs scheduled for that minute.

crontab → The configuration file (CRONjob TABle) that defines
          what to run and when.

Location of crontab files:
  /etc/cron.d/          → System crontabs (for packages/services)
  /etc/cron.hourly/     → Scripts that run hourly (drop files here)
  /etc/cron.daily/      → Scripts that run daily
  /etc/cron.weekly/     → Scripts that run weekly
  /etc/cron.monthly/    → Scripts that run monthly
  /var/spool/cron/crontabs/ → Per-user crontabs (managed by crontab command)

Commands:
  crontab -l   → List your cron jobs
  crontab -e   → Edit your crontab (opens editor)
  crontab -r   → Remove your entire crontab (CAREFUL!)
  crontab -u username -l   → List another user's crontab (root only)
```

The crontab Syntax

```
CRONTAB FORMAT:
═══════════════════════════════════════════════════════════

  ┌─────────────── minute (0-59)
  │  ┌──────────── hour (0-23)
  │  │  ┌───────── day of month (1-31)
  │  │  │  ┌────── month (1-12 or jan-dec)
  │  │  │  │  ┌── day of week (0-7, both 0 and 7 = Sunday, or sun-sat)
  │  │  │  │  │
  *  *  *  *  *  /path/to/command arg1 arg2

SPECIAL CHARACTERS:
  *     → Every (any value)
  ,     → List separator: 1,3,5 means 1 and 3 and 5
  -     → Range: 1-5 means 1,2,3,4,5
  /     → Step: */5 means every 5; 0-30/10 means 0,10,20,30
  ?     → (some implementations) Any value (like *)

EXAMPLES READING LEFT TO RIGHT:
  Minute  Hour  Day  Month  Weekday  Command
  ______  ____  ___  _____  _______  _______
  30      2     *    *      *        → At 02:30 every day
  0       */6   *    *      *        → Every 6 hours, on the hour
  */15    *     *    *      *        → Every 15 minutes
  0       9     *    *      1-5      → 9:00 AM Mon-Fri (weekdays)
  0       0     1    *      *        → Midnight on 1st of every month
  0       0     *    *      0        → Midnight every Sunday
  0       8     1    1      *        → 8:00 AM on January 1st
```

Practical crontab Examples

```
# EDIT YOUR CRONTAB:
crontab -e

# ─── EXAMPLE CRONTAB FILE ──────────────────────────────────
# Format: m h  dom mon dow  command

# IMPORTANT: Set PATH so commands are found
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin
MAILTO=admin@example.com   # Send output/errors to this email

# ─────────────────────────────────────────────────────────
# Every minute (testing only!)
* * * * * /home/alice/scripts/test.sh

# Every 5 minutes
*/5 * * * * /home/alice/scripts/check_service.sh

# Every 15 minutes
*/15 * * * * /home/alice/scripts/sync_data.sh

# Every hour at minute 0
0 * * * * /home/alice/scripts/hourly_report.sh

# Every day at 2:30 AM (great for backups — low traffic time)
30 2 * * * /home/alice/scripts/nightly_backup.sh

# Every weekday (Monday-Friday) at 9:00 AM
0 9 * * 1-5 /home/alice/scripts/send_daily_report.sh
```

```
# Every Monday at 6:00 AM
0 6 * * 1 /home/alice/scripts/weekly_cleanup.sh

# First of every month at midnight
0 0 1 * * /home/alice/scripts/monthly_invoice.sh

# Every 6 hours
0 */6 * * * /home/alice/scripts/sync_to_cloud.sh

# Twice a day: 8 AM and 8 PM
0 8,20 * * * /home/alice/scripts/twice_daily.sh

# At 11:59 PM on December 31
59 23 31 12 * /home/alice/scripts/new_year_cleanup.sh

# ────────────────────────────────────────────────────────────
# WITH OUTPUT REDIRECTION (capture output):
30 2 * * * /home/alice/scripts/backup.sh >> /var/log/backup.log 2>&1

# Suppress all output (silent):
*/5 * * * * /home/alice/scripts/silent_task.sh > /dev/null 2>&1

# Log with timestamp:
0 * * * * /home/alice/scripts/hourly.sh >> /var/log/hourly_$(date +\%Y\%m\%d).log 2>&1
# Note: % must be escaped as \% in crontab!
```

```
READING CRON TIMES — CHEAT CARD:
═══════════════════════════════════════════════════════════

  Want to run...              Write this:
  ____________                __________

  Every minute                * * * * *
  Every 5 minutes             */5 * * * *
  Every 10 minutes            */10 * * * *
  Every 30 minutes            */30 * * * *
  Every hour                  0 * * * *
  Every 2 hours               0 */2 * * *
  Every day at midnight       0 0 * * *
  Every day at 3:30 AM        30 3 * * *
  Every weekday at 9 AM       0 9 * * 1-5
  Every weekend at noon       0 12 * * 6,0
  Every Monday                0 0 * * 1
  Every Sunday at 11 PM       0 23 * * 0
  1st of every month          0 0 1 * *
  Every January 1st           0 0 1 1 *
  Every 15 min during work    */15 9-17 * * 1-5
```

Special cron Schedules

```
# Shorthand keywords (not available in all cron implementations):
@reboot     # Run once at system startup
@yearly     # Run once a year (same as: 0 0 1 1 *)
@annually   # Same as @yearly
@monthly    # Run once a month (same as: 0 0 1 * *)
@weekly     # Run once a week (same as: 0 0 * * 0)
@daily      # Run once a day (same as: 0 0 * * *)
@midnight   # Same as @daily
@hourly     # Run once an hour (same as: 0 * * * *)

# EXAMPLES:
@reboot  /home/alice/scripts/startup.sh         # On every boot
@daily   /home/alice/scripts/cleanup.sh         # Once per day
@weekly  /home/alice/scripts/weekly_report.sh   # Once per week
@monthly /home/alice/scripts/monthly_backup.sh  # Once per month
```

cron for System-Wide Jobs

```
# System crontab (/etc/crontab) has an extra "user" field:
# m  h  dom  mon  dow  USER   command
  30 2  *    *    *    root   /usr/local/bin/backup.sh
  0  6  *    *    1    alice  /home/alice/scripts/weekly.sh

# Drop script files in these directories for easy scheduling:
# /etc/cron.hourly/   → Shell scripts here run EVERY HOUR
# /etc/cron.daily/    → Scripts here run EVERY DAY (run-parts)
# /etc/cron.weekly/   → Scripts here run EVERY WEEK
# /etc/cron.monthly/  → Scripts here run EVERY MONTH

# Requirements for drop-in scripts:
# 1. No file extension (NOT backup.sh — just backup)
# 2. Must be executable (chmod +x)
# 3. Must not have a dot in the name

sudo cp myscript /etc/cron.daily/myscript
sudo chmod +x /etc/cron.daily/myscript
```

The `at` Command

The `at` command schedules a **one-time** task (unlike cron which repeats).

```bash
# Install at (if not present):
sudo apt install at      # Debian/Ubuntu
sudo dnf install at      # RHEL/Fedora

# BASIC USAGE:
at 02:30
# Opens interactive prompt — type commands, then Ctrl+D to save

at now + 5 minutes
at midnight
at noon
at 09:30 tomorrow
at 09:30 next monday
at 09:30 July 31
at 09:30 07/31/2025

# RUN A SCRIPT WITH at:
at 02:30 < /home/alice/scripts/one_time_backup.sh
echo "/home/alice/scripts/one_time_backup.sh" | at 02:30
at 02:30 -f /home/alice/scripts/one_time_backup.sh

# MANAGING at JOBS:
atq         # List pending at jobs (same as: at -l)
atrm 3      # Remove job number 3 (same as: at -d 3)
at -c 3     # Show the content/commands of job 3

# EXAMPLE: Schedule a reminder
echo "notify-send 'Reminder' 'Team standup in 5 minutes'" | at 09:55
```

```
at vs. cron:
═══════════════════════════════════════════════════════════

  CRON                          AT
  ___________________           ___________________

  Repeating tasks               One-time tasks
  "Every day at 2 AM"           "In 30 minutes from now"
  "Every Monday at 9 AM"        "At 3 PM on Tuesday"
  Use crontab -e to configure   Use at command directly
  Jobs persist until removed    Job removed after running
```

Logging cron Output

cron jobs run silently by default. Capture output for debugging.

```
# ─── REDIRECT CRON OUTPUT ──────────────────────────────────

# Append stdout and stderr to a log file:
30 2 * * * /scripts/backup.sh >> /var/log/backup.log 2>&1

# Log with datestamp in filename (% must be escaped):
30 2 * * * /scripts/backup.sh >> /var/log/backup_\%Y\%m\%d.log 2>&1

# Log stdout to file, stderr to different file:
30 2 * * * /scripts/backup.sh >> /var/log/backup.log 2>> /var/log/backup_errors.log

# Suppress all output (jobs that should be silent):
*/5 * * * * /scripts/monitor.sh > /dev/null 2>&1

# ─── BUILD LOGGING INTO YOUR SCRIPT ────────────────────────
#!/bin/bash
# backup_with_logging.sh

LOG_FILE="/var/log/backup.log"
TIMESTAMP=$(date '+%Y-%m-%d %H:%M:%S')

log() {
    echo "[$TIMESTAMP] $*" >> "$LOG_FILE"
}

log "=== Backup started ==="
log "Backing up /var/www..."

if tar -czf "/backup/www_${TIMESTAMP//[: ]/_}.tar.gz" /var/www; then
    log "Backup SUCCESS"
else
```

```
    log "Backup FAILED with exit code $?"
    exit 1
fi

log "=== Backup complete ==="

# ─── VIEW CRON SYSTEM LOG ──────────────────────────────────
# See which cron jobs ran and if they had errors:
grep CRON /var/log/syslog              # Ubuntu/Debian
grep CRON /var/log/cron               # RHEL/Fedora
journalctl -u cron --since "1 hour ago"   # systemd systems
```

A Complete Production-Ready Cron Script

```bash
#!/bin/bash
# nightly_backup.sh — Production backup script for cron
# Schedule: 30 2 * * * /opt/scripts/nightly_backup.sh

set -euo pipefail

# ─── CONFIGURATION ──────────────────────────────────────────
readonly BACKUP_SOURCE="/var/www/html"
readonly BACKUP_DEST="/mnt/backups"
readonly LOG_FILE="/var/log/nightly_backup.log"
readonly MAX_BACKUPS=7          # Keep only last 7 backups
readonly TIMESTAMP=$(date +%Y%m%d_%H%M%S)
readonly BACKUP_FILE="${BACKUP_DEST}/backup_${TIMESTAMP}.tar.gz"

# ─── LOGGING ────────────────────────────────────────────────
log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $*" | tee -a "$LOG_FILE"
}

error() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] ERROR: $*" | tee -a "$LOG_FILE" >&2
}

# ─── CLEANUP ────────────────────────────────────────────────
cleanup() {
    local exit_code=$?
    if [ $exit_code -ne 0 ]; then
        error "Script failed with exit code: $exit_code"
    fi
```

```
    log "Script exiting."
}
trap cleanup EXIT

# ── VALIDATION ──────────────────────────────────────────
[ -d "$BACKUP_SOURCE" ] || { error "Source not found: $BACKUP_SOURCE"; exit 1; }
[ -d "$BACKUP_DEST" ]   || { error "Backup dest not found: $BACKUP_DEST"; exit 1; }

# ── RUN BACKUP ──────────────────────────────────────────
log "=== Starting nightly backup ==="
log "Source: $BACKUP_SOURCE"
log "Destination: $BACKUP_FILE"

if tar -czf "$BACKUP_FILE" "$BACKUP_SOURCE"; then
    SIZE=$(du -sh "$BACKUP_FILE" | cut -f1)
    log "Backup SUCCESS — Size: $SIZE"
else
    error "Backup FAILED"
    exit 1
fi

# ── ROTATE OLD BACKUPS ──────────────────────────────────
log "Rotating backups (keeping last $MAX_BACKUPS)..."
ls -t "${BACKUP_DEST}"/backup_*.tar.gz \
    | tail -n +$(( MAX_BACKUPS + 1 )) \
    | xargs -r rm -v >> "$LOG_FILE" 2>&1

REMAINING=$(ls "${BACKUP_DEST}"/backup_*.tar.gz | wc -l)
log "Rotation complete. $REMAINING backups retained."
```

```
log "=== Backup complete ==="
exit 0
```

---

systemd Timers

Modern Linux systems use systemd timers as a powerful alternative to cron.

```
CRON vs. SYSTEMD TIMERS:
═══════════════════════════════════════════════════════════

CRON:                           SYSTEMD TIMERS:
  Simple, single-file config      Separate service + timer files
  No dependency tracking          Full systemd dependency support
  Hard to debug                   Excellent logging (journalctl)
  Limited error handling          Restart policies, notifications
  Unix standard — everywhere      systemd systems only

USE CRON WHEN:  Simple scripts, maximum compatibility
USE SYSTEMD WHEN: Complex dependencies, need logging, modern systems
```

```
# Step 1: Create the service file
# /etc/systemd/system/backup.service
[Unit]
Description=Nightly Backup Service
After=network.target

[Service]
Type=oneshot
User=root
ExecStart=/opt/scripts/nightly_backup.sh
StandardOutput=journal
StandardError=journal
```

```
# Step 2: Create the timer file
# /etc/systemd/system/backup.timer
[Unit]
Description=Run backup every night at 2:30 AM

[Timer]
OnCalendar=*-*-* 02:30:00   # Every day at 02:30
Persistent=true              # Run if missed (e.g., system was off)

[Install]
WantedBy=timers.target
```

```
# Step 3: Enable and start
sudo systemctl daemon-reload
sudo systemctl enable backup.timer
sudo systemctl start backup.timer

# Check status:
sudo systemctl status backup.timer
sudo systemctl list-timers --all

# View logs:
journalctl -u backup.service
journalctl -u backup.service --since "yesterday"
```

Hands-on Exercise: Complete Cron Setup

```
# ═══ EXERCISE: SET UP A MONITORING CRON JOB ═══

# Step 1: Create the monitoring script
cat > ~/scripts/disk_monitor.sh << 'EOF'
#!/bin/bash
# disk_monitor.sh — Alert if disk usage is too high
# Cron: */30 * * * * /home/$USER/scripts/disk_monitor.sh

THRESHOLD=85
LOG_FILE="/tmp/disk_monitor.log"
TIMESTAMP=$(date '+%Y-%m-%d %H:%M:%S')

usage=$(df / | tail -1 | awk '{print $5}' | tr -d '%')

if [ "$usage" -ge "$THRESHOLD" ]; then
    echo "[$TIMESTAMP] ALERT: Disk usage at ${usage}% (threshold: ${THRESHOLD}%)" \
        | tee -a "$LOG_FILE"
    # In production: send email, Slack alert, PagerDuty, etc.
else
    echo "[$TIMESTAMP] OK: Disk usage at ${usage}%" >> "$LOG_FILE"
fi
EOF

chmod +x ~/scripts/disk_monitor.sh

# Step 2: Test it manually first!
~/scripts/disk_monitor.sh

# Step 3: Add to crontab (runs every 30 minutes)
```

```
(crontab -l 2>/dev/null; echo "*/30 * * * * /home/$USER/scripts/disk_monitor.sh >> /tmp/disk_monitor.log 2>&1") | crontab -

# Step 4: Verify it was added
crontab -l

# Step 5: Check the log after 30 minutes
cat /tmp/disk_monitor.log
```

Section 2.0

File Searching and Archiving

```
╔═══════════════════════════════════════════════════════════╗
║                                                           ║
║   FILE MANAGEMENT MASTERY                                 ║
║                                                           ║
║   Tools you'll master:                                    ║
║   🔍 find   — Search files by any attribute               ║
║   📍 locate — Fast filename database search               ║
║   📦 tar    — Archive files and directories              ║
║   🗜 gzip   — Compress files                              ║
║                                                           ║
╚═══════════════════════════════════════════════════════════╝
```

🎯 Section Learning Objectives

- Use `find` to locate files by name, size, type, date, and permissions
- Execute commands on found files with `find -exec`
- Use `locate` for instant file name searching
- Create and extract archives with `tar`
- Compress and decompress files with `gzip`, `bzip2`, and `xz`
- Build real-world archiving workflows

Section 2a

Using find, locate, tar, gzip

The `find` Command

`find` is the most powerful file search tool in Linux. It searches the filesystem in real-time and can filter by virtually any file attribute.

Basic find Syntax

```
FIND SYNTAX:
═══════════════════════════════════════════════════════════

find  [WHERE to search]  [WHAT to search for]  [WHAT to do with results]

  find  /home/alice      -name "*.log"          -delete
  find  /var/log         -size +10M             -ls
  find  .                -mtime -7              -print
  find  /etc             -type f -name "*.conf" -exec cat {} \;

WHERE: Path to search (. = current dir, / = entire system)
WHAT:  Tests/filters (name, size, type, time, permissions, etc.)
DO:    Action on results (-print, -ls, -delete, -exec, -ok)
```

Finding by Name

```bash
# EXACT NAME:
find / -name "passwd"               # Find file named exactly "passwd"
find / -name "passwd" 2>/dev/null   # Suppress "permission denied" errors

# CASE-INSENSITIVE NAME:
find / -iname "readme.md"           # Finds README.md, readme.MD, Readme.md

# WILDCARDS IN NAME:
find /var/log -name "*.log"         # All .log files
find /home -name "*.txt"            # All .txt files in /home
find . -name "backup*"              # Files starting with "backup"
find . -name "*2024*"               # Files with "2024" in name
find . -name "report_??_2024.pdf"   # ? matches exactly one character

# MULTIPLE PATTERNS:
find . -name "*.jpg" -o -name "*.png" -o -name "*.gif"   # jpg OR png OR gif
find . \( -name "*.jpg" -o -name "*.png" \) -size +1M    # Images > 1MB

# EXCLUDE A PATH:
find /home -name "*.log" -not -path "*/archive/*"        # Skip archive dirs
find / -name "*.conf" -not -path "*/proc/*"              # Skip /proc

# FIND BY PATH (directory component):
find / -path "*/nginx/*.conf"        # conf files inside nginx directories
find . -path "*/node_modules"        # Find node_modules directories
```

Finding by Type

```
# FILE TYPES:
find /home -type f    # Regular files only
find /home -type d    # Directories only
find /home -type l    # Symbolic links only
find /home -type b    # Block devices
find /home -type c    # Character devices
find /home -type p    # Named pipes (FIFOs)
find /home -type s    # Sockets

# PRACTICAL EXAMPLES:
# Find all directories named "cache":
find / -type d -name "cache" 2>/dev/null

# Find all broken symbolic links:
find /usr -type l -! -exec test -e {} \; -print
# Better:
find /usr -xtype l    # -xtype l = symbolic link pointing to nothing

# Find all executable files:
find /usr/local/bin -type f -executable

# Find all empty files:
find /tmp -type f -empty

# Find all empty directories:
find /home -type d -empty
```

Finding by Size

```
# SIZE UNITS:
# c = bytes, k = kilobytes, M = megabytes, G = gigabytes

# EXACT SIZE (rarely used):
find . -size 100k       # Exactly 100 512-byte blocks (not what you think!)

# GREATER THAN:
find . -size +100M      # Files larger than 100MB
find /var -size +500M   # Files over 500MB in /var
find / -size +1G        # Files over 1 gigabyte

# LESS THAN:
find . -size -10k       # Files smaller than 10KB

# RANGE:
find . -size +1M -size -100M    # Between 1MB and 100MB

# FIND LARGE FILES — most common use case:
find / -size +100M -type f 2>/dev/null \
    | xargs du -sh 2>/dev/null \
    | sort -rh \
    | head -20

# FIND AND DISPLAY SIZE:
find /home -size +10M -type f -exec du -sh {} \;
```

Finding by Time

```
# TIME OPTIONS:
# -mtime  = modification time (content last changed)
# -atime  = access time (last read)
# -ctime  = change time (metadata/permissions last changed)
# -newer file = newer than reference file

# TIME VALUES:
# +N = MORE than N days ago
# -N = LESS than N days ago (i.e., within last N days)
#  N = EXACTLY N days ago

# FILES MODIFIED WITHIN LAST 24 HOURS:
find . -mtime -1           # Less than 1 day old
find . -mtime 0            # Modified today

# FILES MODIFIED WITHIN LAST 7 DAYS:
find /home -mtime -7 -type f

# FILES NOT MODIFIED IN LAST 30 DAYS (old files):
find /tmp -mtime +30 -type f

# FILES OLDER THAN 90 DAYS:
find /var/log -name "*.log" -mtime +90

# MINUTES-BASED TIME (-mmin, -amin, -cmin):
find /tmp -mmin -60        # Modified in last 60 minutes
find . -mmin +120          # Modified more than 2 hours ago

# NEWER THAN A REFERENCE FILE:
```

```
touch reference_point
# ... wait, create files, do work ...
find . -newer reference_point   # Files created/modified after reference
```

Finding by Permissions and Owner

```
# BY PERMISSIONS:
find / -perm 777           # Exactly rwxrwxrwx (dangerous!)
find / -perm -777          # At least rwxrwxrwx (all 3 bits set)
find / -perm /777          # ANY of r, w, or x set for any user
find / -perm -o+w          # World-writable (security issue!)
find / -perm -4000         # SUID bit set (potential security issue)
find / -perm -2000         # SGID bit set
find / -perm -1000         # Sticky bit set

# SECURITY AUDIT — find dangerous permissions:
find / -perm -4000 -type f 2>/dev/null    # All SUID files
find / -perm -o+w -type f 2>/dev/null     # World-writable files
find / -perm 777 -type f 2>/dev/null      # Full-permission files

# BY OWNER:
find /home -user alice          # Files owned by alice
find /home -user 1001           # Files owned by UID 1001
find /home -group developers    # Files owned by group developers
find /home -nouser              # Files with no valid owner (orphaned)
find /home -nogroup             # Files with no valid group

# COMBINATION:
find /var/www -user www-data -type f -name "*.php"   # PHP files owned by www-data
```

Executing Commands on Found Files (`-exec`)

This is where `find` becomes incredibly powerful:

```bash
# ─── -exec SYNTAX ──────────────────────────────────────────
# {} = placeholder for each found file
# \; = end of command (one command per file — slower)
# +  = end of command (batch — passes multiple files, faster)

# Print found files:
find . -name "*.log" -exec echo "Found: {}" \;

# Delete found files (CAREFUL!):
find /tmp -mtime +30 -type f -exec rm {} \;
# Safer — asks confirmation:
find /tmp -mtime +30 -type f -ok rm {} \;     # -ok asks y/n for each

# Change permissions on found files:
find /var/www -type f -exec chmod 644 {} +
find /var/www -type d -exec chmod 755 {} +

# Compress old log files:
find /var/log -name "*.log" -mtime +7 -exec gzip {} \;

# Copy found files:
find . -name "*.conf" -exec cp {} /backup/ \;

# Search within found files (grep through find results):
find . -name "*.py" -exec grep -l "import os" {} \;
# OR faster with +:
find . -name "*.py" -exec grep -l "import os" {} +

# Run multiple commands (use sh -c):
find . -name "*.log" -exec sh -c '
    echo "Processing: $1"
```

```
    wc -l "$1"
' _ {} \;

# ─── -execdir (safer than -exec) ──────────────────────────
# Runs command in the directory containing the found file
find . -name "*.tmp" -execdir rm {} \;   # Safer — no path injection risk

# ─── XARGS (alternative to -exec) ─────────────────────────
# xargs is faster for large numbers of files
find . -name "*.log" | xargs wc -l       # Count lines in all logs
find . -name "*.txt" | xargs grep "TODO" # Grep in all txt files
find . -name "*.jpg" | xargs -I{} cp {} /backup/   # -I{} for placeholder

# Handle filenames with spaces (use -print0 and -0):
find . -name "*.txt" -print0 | xargs -0 rm   # Null-delimited for safety
```

Real-World `find` Scripts

```bash
#!/bin/bash
# cleanup_old_files.sh — Clean up files older than 30 days

TARGET_DIR="${1:-/tmp}"
DAYS="${2:-30}"
DRY_RUN="${3:-false}"

echo "Scanning: $TARGET_DIR"
echo "Finding files older than $DAYS days..."
echo "──────────────────────────────────"

# Count total files to clean
count=$(find "$TARGET_DIR" -maxdepth 3 -type f -mtime +"$DAYS" 2>/dev/null | wc -l)
size=$(find "$TARGET_DIR" -maxdepth 3 -type f -mtime +"$DAYS" 2>/dev/null \
       -exec du -ch {} + 2>/dev/null | tail -1 | cut -f1)

echo "Found: $count files ($size)"

if [[ "$DRY_RUN" == "true" ]]; then
    echo "DRY RUN — listing only (no deletion):"
    find "$TARGET_DIR" -maxdepth 3 -type f -mtime +"$DAYS" 2>/dev/null
else
    echo "Deleting..."
    find "$TARGET_DIR" -maxdepth 3 -type f -mtime +"$DAYS" \
        -exec rm -v {} \; 2>/dev/null
    echo "Cleanup complete."
fi
```

```bash
#!/bin/bash
# find_duplicates.sh — Find files with identical names in a directory tree

find "${1:-.}" -type f -printf "%f\n" | sort | uniq -d | while read -r name; do
    echo "Duplicate filename: $name"
    find "${1:-.}" -type f -name "$name" | sed 's/^/  → /'
    echo ""
done
```

Advanced `find` Options

```bash
# LIMIT DEPTH:
find . -maxdepth 2 -name "*.sh"    # Only go 2 levels deep
find . -mindepth 2 -name "*.sh"    # Skip first 2 levels
find . -maxdepth 1 -type f         # Only current directory (no subdirs)

# FOLLOW SYMBOLIC LINKS:
find . -L -type f -name "*.conf"   # -L follows symlinks

# DON'T CROSS FILESYSTEM BOUNDARIES:
find / -xdev -size +1G 2>/dev/null # Only search current filesystem (not NFS/mounts)

# PRINT WITH FORMAT:
find . -name "*.sh" -printf "%p\t%s bytes\t%TY-%Tm-%Td\n"
# %p = path, %s = size, %T = time (formatted)

# PRUNE — skip directories:
find . -name node_modules -prune -o -name "*.js" -print
# Finds .js files but SKIPS any node_modules directories
```

The `locate` Command

`locate` uses a pre-built database to find files instantly — much faster than `find` for simple name searches.

```bash
# INSTALL (if not present):
sudo apt install mlocate    # Debian/Ubuntu
sudo dnf install mlocate    # RHEL/Fedora

# UPDATE THE DATABASE (run as root, or wait for nightly cron):
sudo updatedb

# BASIC SEARCH:
locate passwd               # Find all files with "passwd" in path
locate "*.conf"             # Find all .conf files
locate nginx.conf           # Find nginx.conf anywhere

# CASE-INSENSITIVE:
locate -i readme            # Finds README, readme, Readme

# COUNT ONLY:
locate -c "*.log"           # Count matching files

# SHOW ONLY EXISTING FILES (database may be stale):
locate -e "*.conf"          # -e checks each file actually exists

# LIMIT RESULTS:
locate -l 10 "*.log"        # Show only first 10 matches

# REGEX:
locate -r "\.sh$"           # Files ending in .sh (regex)
locate -r "/etc/.*\.conf$"  # .conf files in /etc

# SHOW DATABASE STATS:
locate --statistics
# Database timestamp: Mon Jan 15 03:22:47 2024
```

```
# Directories: 45,213
# Files: 782,941
# ...
```

---

find vs. locate

```
FIND vs. LOCATE — WHEN TO USE WHICH
═══════════════════════════════════════════════════════════════

FEATURE              FIND                    LOCATE
──────────────────   ────────────────────    ──────────────────

Speed                Slow (real-time scan)   Fast (database)
Freshness            Always current          Database may be stale
Search by name       ✅                       ✅ (main use)
Search by size       ✅                       ❌
Search by time       ✅                       ❌
Search by owner      ✅                       ❌
Search by content    ✅ (with -exec grep)     ❌
Execute commands     ✅                       ❌
Root needed          Sometimes               No
System load          Higher                  Very low

USE find WHEN:
  ✅ Searching by size, date, permissions, owner
  ✅ Results must be 100% current (find is real-time)
  ✅ You need to execute commands on results (-exec)
  ✅ Complex searches with multiple criteria
  ✅ Searching a specific directory tree

USE locate WHEN:
  ✅ You just need to find where a file is
  ✅ You know (part of) the filename
  ✅ Speed matters — searching the whole system
  ✅ Simple name-based lookup
```

```
COMBINED USAGE (best of both):
  locate "*.log" | xargs find -size +10M -type f 2>/dev/null
  # Fast locate to find candidates, then find to filter by size
```

The `tar` Command

`tar` (Tape ARchive) combines multiple files into one archive file. Despite the name, it works on disks, not just tapes.

The tar Mental Model

```
WHAT tar DOES
═══════════════════════════════════════════════════════════

CREATING AN ARCHIVE:
  file1.txt  ─┐
  file2.txt  ─┤  tar  →  archive.tar  (one file, preserves structure)
  mydir/     ─┘

CREATING A COMPRESSED ARCHIVE:
  file1.txt  ─┐
  file2.txt  ─┤  tar + gzip  →  archive.tar.gz  (compressed!)
  mydir/     ─┘

EXTRACTING:
  archive.tar.gz  →  tar  →  file1.txt, file2.txt, mydir/

EXTENSIONS:
  .tar        → Archive only (no compression)
  .tar.gz     → Archive + gzip compression (most common)
  .tgz        → Same as .tar.gz (shorter extension)
  .tar.bz2    → Archive + bzip2 compression (better, slower)
  .tar.xz     → Archive + xz compression (best, slowest)
  .tar.zst    → Archive + zstd compression (modern, fast+good)
```

Core tar Flags

```
TAR FLAGS REFERENCE:
═══════════════════════════════════════════════════════════


OPERATIONS (choose one):
  -c   Create a new archive
  -x   Extract from archive
  -t   List contents of archive (don't extract)
  -r   Append files to existing archive
  -u   Update archive (add files newer than archive)

COMPRESSION (combine with operation):
  -z   Use gzip   (.tar.gz or .tgz)
  -j   Use bzip2  (.tar.bz2)
  -J   Use xz     (.tar.xz)
  --zstd  Use zstd (.tar.zst)

GENERAL OPTIONS:
  -v   Verbose (list files as processed)
  -f   File — specifies the archive filename (ALWAYS use!)
  -p   Preserve file permissions
  -P   Preserve absolute paths (keep leading /)
  -C   Change to directory before extracting

HANDY COMBINATIONS:
  -cvf    Create, Verbose, File
  -xvf    Extract, Verbose, File
  -tvf    Table of contents, Verbose, File
```

```
-czvf   Create, gzip, Verbose, File
-xzvf   Extract, gzip, Verbose, File
```

Creating Archives

```bash
# ─── CREATE UNCOMPRESSED ARCHIVE ──────────────────────────────
tar -cvf archive.tar file1.txt file2.txt mydir/
# -c = create, -v = verbose, -f = filename
# Creates: archive.tar

# ─── CREATE WITH COMPRESSION ──────────────────────────────────
# gzip (fast, moderate compression):
tar -czvf archive.tar.gz mydir/
# -z = gzip compression
# Creates: archive.tar.gz

# bzip2 (slower, better compression):
tar -cjvf archive.tar.bz2 mydir/
# -j = bzip2 compression

# xz (slowest, best compression):
tar -cJvf archive.tar.xz mydir/
# -J = xz compression

# ─── PRACTICAL EXAMPLES ───────────────────────────────────────
# Archive entire project directory:
tar -czvf project_backup.tar.gz /home/alice/myproject/

# Archive with timestamp in filename:
tar -czvf "backup_$(date +%Y%m%d_%H%M%S).tar.gz" /var/www/html/

# Archive multiple sources:
tar -czvf backup.tar.gz /etc/nginx /etc/ssl /var/www
```

```bash
# Archive excluding certain files/directories:
tar -czvf backup.tar.gz /var/www/ \
    --exclude="*.log" \
    --exclude="*/node_modules/*" \
    --exclude="*.tmp"

# Archive excluding from a file:
cat > exclude.txt << EOF
*.log
node_modules/
*.tmp
.git/
EOF
tar -czvf backup.tar.gz /var/www/ --exclude-from=exclude.txt

# Test archive integrity after creation:
tar -tzvf backup.tar.gz | head -20   # List first 20 entries
```

Listing Archive Contents

```
# LIST (without extracting):
tar -tvf archive.tar           # List uncompressed
tar -tzvf archive.tar.gz       # List gzip-compressed
tar -tjvf archive.tar.bz2      # List bzip2-compressed
tar -tJvf archive.tar.xz       # List xz-compressed

# tar auto-detects compression with --auto-compress:
tar -tvf archive.tar.gz        # Works — tar detects gzip from extension

# SAMPLE OUTPUT:
# drwxr-xr-x alice/alice   0 2024-01-15 09:23 myproject/
# -rw-r--r-- alice/alice 1024 2024-01-15 09:20 myproject/README.md
# -rw-r--r-- alice/alice 4096 2024-01-15 09:22 myproject/app.js

# JUST FILENAMES (no verbose):
tar -tf archive.tar.gz          # Clean list of paths

# COUNT FILES:
tar -tf archive.tar.gz | wc -l
```

Extracting Archives

```
# ─── EXTRACT EVERYTHING ──────────────────────────────────────
tar -xvf archive.tar                # Extract uncompressed
tar -xzvf archive.tar.gz            # Extract gzip
tar -xjvf archive.tar.bz2           # Extract bzip2
tar -xJvf archive.tar.xz            # Extract xz

# Auto-detect compression (works for most formats):
tar -xvf archive.tar.gz             # tar auto-detects!
tar xf archive.tar.gz               # Shortest form (no dashes)

# EXTRACT TO SPECIFIC DIRECTORY:
tar -xzvf archive.tar.gz -C /tmp/restore/   # -C = change to directory
tar -xzvf archive.tar.gz -C /opt/

# EXTRACT SPECIFIC FILES:
tar -xzvf archive.tar.gz myproject/README.md
tar -xzvf archive.tar.gz myproject/config/

# EXTRACT AND STRIP LEADING DIRECTORY:
tar -xzvf archive.tar.gz --strip-components=1
# If archive has myproject/file.txt, extracts as file.txt (strips "myproject/")

# EXTRACT PRESERVING PERMISSIONS:
tar -xzvpf archive.tar.gz    # -p preserves permissions, ownership

# ─── REAL-WORLD EXTRACT PATTERNS ─────────────────────────────
# Preview before extracting:
tar -tzvf backup.tar.gz | less   # Browse contents, then:
tar -xzvf backup.tar.gz -C /tmp/preview/
```

```
# Safe extraction (always extract to subdirectory, don't clobber root):
mkdir -p restore_$(date +%Y%m%d)
tar -xzvf backup.tar.gz -C restore_$(date +%Y%m%d)/ --strip-components=1
```

Complete tar Reference

```
# ─── QUICK REFERENCE ──────────────────────────────────────

# CREATE:
tar -czvf output.tar.gz input-dir/       # With gzip
tar -cjvf output.tar.bz2 input-dir/     # With bzip2
tar -cJvf output.tar.xz input-dir/      # With xz

# LIST CONTENTS:
tar -tzvf archive.tar.gz

# EXTRACT:
tar -xzvf archive.tar.gz                 # To current dir
tar -xzvf archive.tar.gz -C /dest/       # To specific dir

# EXCLUDE:
tar -czvf out.tar.gz dir/ --exclude="*.log"

# VERIFY:
tar -tzvf archive.tar.gz > /dev/null && echo "Archive OK"
```

Compression: gzip, bzip2, xz

While `tar` archives files, compression tools make them smaller.

gzip

```
# COMPRESS a file (replaces original with .gz):
gzip filename.txt             # Creates filename.txt.gz, removes original
gzip -k filename.txt          # -k = keep original (don't delete)
gzip -9 filename.txt          # -9 = maximum compression (slower)
gzip -1 filename.txt          # -1 = fastest (least compression)
gzip *.log                    # Compress all .log files

# DECOMPRESS:
gzip -d filename.txt.gz       # Decompress, removes .gz
gunzip filename.txt.gz        # Same as gzip -d

# DECOMPRESS to stdout (don't remove .gz):
gzip -dc filename.txt.gz      # -d=decompress, -c=stdout (keep .gz)
zcat filename.txt.gz          # Same — print contents

# VIEW compressed file without decompressing:
zcat logfile.gz | grep ERROR
zcat logfile.gz | head -20
zless logfile.gz              # Browse with less
zgrep "pattern" logfile.gz    # grep in gzip file

# CHECK COMPRESSION RATIO:
gzip -l archive.tar.gz
#          compressed        uncompressed  ratio uncompressed_name
#              45872              163840  72.0% archive.tar

# MULTIPLE FILES:
```

```
gzip file1.txt file2.txt      # Each gets its own .gz
gzip -r /path/to/dir/         # Recursively compress directory
```

bzip2

```
# bzip2 — better compression than gzip, slower

# COMPRESS:
bzip2 filename.txt            # Creates filename.txt.bz2
bzip2 -k filename.txt         # Keep original
bzip2 -9 filename.txt         # Maximum compression

# DECOMPRESS:
bzip2 -d filename.txt.bz2
bunzip2 filename.txt.bz2      # Same as bzip2 -d

# VIEW without decompressing:
bzcat filename.bz2 | grep pattern
bzless filename.bz2
bzgrep "pattern" filename.bz2
```

xz

```
# xz — best compression ratio, slowest

# COMPRESS:
xz filename.txt              # Creates filename.txt.xz
xz -k filename.txt           # Keep original
xz -9 filename.txt           # Maximum compression
xz -T 0 filename.txt         # Use all CPU threads (parallel, faster)

# DECOMPRESS:
xz -d filename.txt.xz
unxz filename.txt.xz         # Same as xz -d

# VIEW:
xzcat filename.xz | grep pattern
xzless filename.xz

# CHECK COMPRESSION RATIO:
xz --list archive.tar.xz
```

Compression Comparison

```
COMPRESSION COMPARISON:
═══════════════════════════════════════════════════════════

Test: 100MB application directory

  Tool     Extension    Size     Time     Ratio    Best Use
  ______   ___________  ______   ______   ______   ____________________
  (none)   .tar         100 MB   0.1s     1.0x     When speed is only goal
  gzip     .tar.gz      28 MB    3s       3.6x     General use, MOST COMMON
  bzip2    .tar.bz2     22 MB    12s      4.5x     Better compression needed
  xz       .tar.xz      18 MB    45s      5.6x     Best compression (releases)
  zstd     .tar.zst     24 MB    1s       4.2x     Modern: fast + good (new)

RECOMMENDATION:
  📦 Everyday backups:      .tar.gz  (gzip — fast, universal)
  📮 Software distribution: .tar.xz  (xz — best compression for releases)
  🚀 Speed-critical:        .tar.zst (zstd — modern, fastest good ratio)
  🌐 Maximum compatibility: .tar.gz  (gzip is on everything)
```

`zip` and `unzip` (for Windows Compatibility)

```
# zip is common on Windows — useful for cross-platform sharing
zip archive.zip file1.txt file2.txt
zip -r archive.zip mydir/           # Recursive (include directory)
zip -9 archive.zip files/           # Maximum compression
zip -e archive.zip file.txt         # Encrypt with password

# Unzip:
unzip archive.zip                   # Extract to current directory
unzip archive.zip -d /dest/dir/     # Extract to specific directory
unzip -l archive.zip                # List contents without extracting
unzip -t archive.zip                # Test archive integrity

# View without extracting:
zipinfo archive.zip                 # Detailed info
unzip -p archive.zip file.txt       # Print specific file to stdout
```

Practical Archiving Workflows

Workflow 1: Complete Backup System

```bash
#!/bin/bash
# backup_system.sh — Full system backup script

set -euo pipefail

# ─── CONFIGURATION ──────────────────────────────────────────
BACKUP_SOURCES=(
    "/etc"
    "/home"
    "/var/www"
    "/var/lib/mysql"   # Only if MySQL is installed
)
BACKUP_DEST="/mnt/backups"
RETENTION_DAYS=30
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
LOG_FILE="/var/log/backup_${TIMESTAMP}.log"

# ─── LOGGING ────────────────────────────────────────────────
log() { echo "[$(date '+%H:%M:%S')] $*" | tee -a "$LOG_FILE"; }

# ─── BACKUP FUNCTION ────────────────────────────────────────
backup_path() {
    local source="$1"
    local name
    name=$(echo "$source" | tr '/' '_' | sed 's/^_//')
    local dest="${BACKUP_DEST}/${name}_${TIMESTAMP}.tar.gz"
```

```
    if [ ! -e "$source" ]; then
        log "SKIP: $source does not exist"
        return 0
    fi

    log "Backing up: $source → $(basename "$dest")"
    if tar -czvf "$dest" "$source" \
        --exclude="*.tmp" \
        --exclude="*/cache/*" \
        2>> "$LOG_FILE"; then
        local size
        size=$(du -sh "$dest" | cut -f1)
        log "   ✅ Success — Size: $size"
    else
        log "   ❌ FAILED: $source"
        return 1
    fi
}

# ── CLEANUP OLD BACKUPS ──────────────────────────────────
rotate_backups() {
    log "Rotating backups older than $RETENTION_DAYS days..."
    local count
    count=$(find "$BACKUP_DEST" -name "*.tar.gz" -mtime +"$RETENTION_DAYS" | wc -l)
    find "$BACKUP_DEST" -name "*.tar.gz" -mtime +"$RETENTION_DAYS" -delete
    log "   Removed $count old backup(s)"
}

# ── MAIN ─────────────────────────────────────────────────
log "═══════════════════════════════"
log "BACKUP STARTED: $TIMESTAMP"
```

```
log "═══════════════════════════════"

# Ensure backup destination exists
mkdir -p "$BACKUP_DEST" || { log "Cannot create $BACKUP_DEST"; exit 1; }

# Run backups
ERRORS=0
for source in "${BACKUP_SOURCES[@]}"; do
    backup_path "$source" || (( ERRORS++ )) || true
done

# Rotate old backups
rotate_backups

# Summary
log "═══════════════════════════════"
log "BACKUP COMPLETE — Errors: $ERRORS"
log "Log: $LOG_FILE"
log "═══════════════════════════════"

[ $ERRORS -eq 0 ] || exit 1
```

Workflow 2: Find and Archive Old Logs

```bash
#!/bin/bash
# archive_old_logs.sh — Find and compress log files older than N days

LOG_DIR="${1:-/var/log}"
ARCHIVE_DIR="${2:-/var/log/archive}"
AGE_DAYS="${3:-7}"

mkdir -p "$ARCHIVE_DIR"

echo "Archiving logs older than $AGE_DAYS days from $LOG_DIR"
echo "Archive destination: $ARCHIVE_DIR"
echo "━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━"

# Find and compress old logs (but not already-compressed ones)
ARCHIVED=0
while IFS= read -r logfile; do
    if gzip -k "$logfile" 2>/dev/null; then
        mv "${logfile}.gz" "$ARCHIVE_DIR/"
        rm "$logfile"
        echo "✅ Archived: $(basename "$logfile")"
        (( ARCHIVED++ ))
    else
        echo "❌ Failed: $logfile"
    fi
done < <(find "$LOG_DIR" -maxdepth 1 -name "*.log" -mtime +"$AGE_DAYS" -type f)

echo "━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━"
echo "Archived: $ARCHIVED log file(s)"
```

```
# Remove archives older than 90 days
find "$ARCHIVE_DIR" -name "*.gz" -mtime +90 -delete
echo "Removed archives older than 90 days"
```

Workflow 3: Deployment Packaging

```bash
#!/bin/bash
# package_release.sh — Package application for deployment

APP_NAME="myapp"
VERSION="${1:?Usage: $0 <version>}"
BUILD_DIR="./dist"
PACKAGE_DIR="./packages"

echo "Packaging $APP_NAME version $VERSION..."
mkdir -p "$PACKAGE_DIR"

# Build the package
tar -cJvf "${PACKAGE_DIR}/${APP_NAME}-${VERSION}.tar.xz" \
    --transform "s|^${BUILD_DIR#./}|${APP_NAME}-${VERSION}|" \
    --exclude="*.test.js" \
    --exclude="*/__tests__/*" \
    --exclude="*.log" \
    "$BUILD_DIR"

# Generate checksum for verification
sha256sum "${PACKAGE_DIR}/${APP_NAME}-${VERSION}.tar.xz" \
    > "${PACKAGE_DIR}/${APP_NAME}-${VERSION}.tar.xz.sha256"

echo "Package created:"
ls -lh "${PACKAGE_DIR}/${APP_NAME}-${VERSION}.tar.xz"
echo "SHA256:"
cat "${PACKAGE_DIR}/${APP_NAME}-${VERSION}.tar.xz.sha256"
```

Workflow 4: Search-and-Process with find

```bash
#!/bin/bash
# audit_permissions.sh — Find and report files with concerning permissions

echo "═══════════════════════════════════════"
echo "  PERMISSION AUDIT REPORT"
echo "  $(date)"
echo "═══════════════════════════════════════"

# World-writable files (potential security issue)
echo ""
echo "⚠️  WORLD-WRITABLE FILES:"
find /home /var/www /tmp -type f -perm -o+w 2>/dev/null \
    | grep -v "^/tmp" \
    | while read -r f; do
        ls -la "$f"
    done

# SUID files (run as owner, not caller)
echo ""
echo "⚠️  SUID FILES (run as file owner):"
find /usr /bin /sbin -perm -4000 -type f 2>/dev/null | while read -r f; do
    ls -la "$f"
done

# Files with no owner
echo ""
echo "⚠️  FILES WITH NO OWNER:"
find /home -nouser -type f 2>/dev/null
```

```
# Large files (potential disk hogs)
echo ""
echo "📦 LARGE FILES (>500MB):"
find / -size +500M -type f 2>/dev/null \
    | xargs du -sh 2>/dev/null \
    | sort -rh

echo ""
echo "═══════════════════════════════════════"
echo "Audit complete."
```

Complete Cheat Sheet

```
═══════════════════════════════════════════════════════════════
              BASH SCRIPTING CHEAT SHEET
═══════════════════════════════════════════════════════════════

SCRIPT BASICS
  #!/bin/bash           Shebang line — always first
  chmod +x script.sh    Make executable
  ./script.sh           Run with path
  bash script.sh        Run with bash (no chmod needed)
  bash -x script.sh     Debug mode
  bash -n script.sh     Syntax check only

VARIABLES
  name="value"          Assign (no spaces around =!)
  echo "$name"          Use variable
  echo "${name}"        Use variable (explicit)
  readonly VAR=value    Read-only (constant)
  local var=value       Local to function
  unset var             Delete variable

SPECIAL VARIABLES
  $0 $1 $2 ..          Script name, positional args
  $@                   All args (separate)
  $#                   Number of args
  $$                   Current PID
  $?                   Last exit code
  $HOME $USER $PATH    Environment variables
```

```
VARIABLE EXPANSION
  ${var:-default}      Use default if unset/empty
  ${var:=default}      Set and use default if unset
  ${#var}              Length of var
  ${var%suffix}        Remove shortest suffix
  ${var%%suffix}       Remove longest suffix
  ${var#prefix}        Remove shortest prefix
  ${var##prefix}       Remove longest prefix
  ${var/old/new}       Replace first occurrence
  ${var//old/new}      Replace all occurrences
  ${var,,}             Lowercase (Bash 4+)
  ${var^^}             Uppercase (Bash 4+)

CONDITIONALS
  if [ cond ]; then ... fi
  if [[ cond ]]; then ... fi   (bash extended)
  [ -f file ]   File exists and is regular
  [ -d dir ]    Directory exists
  [ -e path ]   Path exists
  [ -r file ]   File is readable
  [ -x file ]   File is executable
  [ -z "$s" ]   String is empty
  [ -n "$s" ]   String is non-empty
  [ "$a" = "$b" ]   String equal
  [ $a -eq $b ]     Numeric equal
  [ $a -lt $b ]     Numeric less than
  [ $a -gt $b ]     Numeric greater than
  &&    AND     ||   OR     !  NOT

LOOPS
  for item in list; do ... done
  for i in {1..10}; do ... done
```

```
  for (( i=0; i<10; i++ )); do ... done
  while [ condition ]; do ... done
  until [ condition ]; do ... done
  break   Exit loop
  continue  Skip to next iteration

FUNCTIONS
  myfunc() { ... }      Define function
  myfunc arg1 arg2      Call function
  local var=value       Local variable
  return 0              Return exit code
  echo "value"          "Return" a value (capture with $())

ARRAYS
  arr=("a" "b" "c")    Create array
  ${arr[0]}            First element
  ${arr[@]}            All elements
  ${#arr[@]}           Array length
  arr+=("d")           Append element
  declare -A map        Associative array (Bash 4+)

ARITHMETIC
  (( result = a + b ))     Integer arithmetic
  $(( a * b ))             Arithmetic expansion
  echo "scale=2; 10/3" | bc   Float arithmetic

STRINGS
  [[ "$s" == pattern ]]   Pattern match
  [[ "$s" =~ regex ]]     Regex match
  ${#s}                   String length

INPUT/OUTPUT
```

```
  read -p "Prompt: " var    Read user input
  read -sp "Password: " pw  Silent read
  echo "text"               Print with newline
  printf "fmt" args         Formatted print
  command > file            Redirect stdout
  command >> file           Append stdout
  command 2> file           Redirect stderr
  command 2>&1              Redirect stderr to stdout
  command > /dev/null 2>&1  Suppress all output

ERROR HANDLING
  set -e              Exit on error
  set -u              Error on undefined var
  set -o pipefail     Pipe fails on any error
  set -euo pipefail   All three (use this!)
  trap 'cleanup' EXIT   Run on exit
  exit 0 / exit 1     Success / failure

═══════════════════════════════════════════════════════════════
                    CRON CHEAT SHEET
═══════════════════════════════════════════════════════════════

CRONTAB COMMANDS
  crontab -l    List cron jobs
  crontab -e    Edit cron jobs
  crontab -r    Remove all cron jobs

FORMAT: m h dom mon dow command
  *  = any value
  ,  = list: 1,3,5
  -  = range: 1-5
  /  = step: */5
```

```
COMMON SCHEDULES
  * * * * *         Every minute
  */5 * * * *       Every 5 minutes
  0 * * * *         Every hour
  0 2 * * *         Daily at 2 AM
  0 9 * * 1-5       Weekdays at 9 AM
  0 0 * * 0         Sundays at midnight
  0 0 1 * *         1st of every month
  @reboot           At system boot
  @daily            Once per day
  @weekly           Once per week

OUTPUT REDIRECTION IN CRON
  command >> /log 2>&1      Capture all output
  command > /dev/null 2>&1  Silent

═══════════════════════════════════════════════════════════════
                    find CHEAT SHEET
═══════════════════════════════════════════════════════════════

find [PATH] [OPTIONS] [ACTION]

BY NAME:
  -name "*.log"       Exact pattern (case sensitive)
  -iname "*.log"      Case insensitive
  -path "*/dir/*"     Match full path

BY TYPE:
  -type f    Regular file
  -type d    Directory
  -type l    Symbolic link
```

```
BY SIZE:
  -size +100M    Larger than 100MB
  -size -10k     Smaller than 10KB
  -size +1G      Larger than 1GB

BY TIME:
  -mtime -7    Modified within 7 days
  -mtime +30   Not modified in 30 days
  -mmin -60    Modified within 60 minutes
  -newer file  Newer than reference file

BY PERMISSIONS:
  -perm 755      Exact permissions
  -perm -o+w     World-writable
  -perm -4000    SUID set

BY OWNER:
  -user alice     Owned by alice
  -group devs     Owned by group devs
  -nouser         No valid owner

DEPTH:
  -maxdepth 2    Don't go deeper than 2 levels
  -mindepth 1    Skip top level

ACTIONS:
  -print         Print path (default)
  -ls            Long listing
  -delete        Delete found files
  -exec cmd {} \;   Run command on each
  -exec cmd {} +    Run command on batch
```

```
  -ok cmd {} \;     Like -exec but confirms

═══════════════════════════════════════════════════════════════
                    tar CHEAT SHEET
═══════════════════════════════════════════════════════════════

CREATE:
  tar -czvf out.tar.gz dir/     gzip  (recommended)
  tar -cjvf out.tar.bz2 dir/    bzip2
  tar -cJvf out.tar.xz dir/     xz (best compression)

LIST:
  tar -tzvf archive.tar.gz      List contents

EXTRACT:
  tar -xzvf archive.tar.gz              To current dir
  tar -xzvf archive.tar.gz -C /dest/   To specific dir
  tar -xzvf archive.tar.gz file.txt    Extract one file

EXCLUDE:
  tar -czvf out.tar.gz dir/ --exclude="*.log"

COMPRESSION TOOLS:
  gzip file.txt              Compress (creates file.txt.gz)
  gzip -d file.txt.gz        Decompress
  gzip -k file.txt           Compress, keep original
  gunzip file.txt.gz         Same as gzip -d
  zcat file.gz               View without decompressing
  zgrep "pat" file.gz        Grep without decompressing

locate:
  updatedb                   Rebuild database
```

```
  locate filename           Search by name
  locate -i FILENAME        Case insensitive
  locate -c "*.log"         Count matches
  locate -e "*.conf"        Only existing files

═══════════════════════════════════════════════════════════
```

Additional Resources

Resource	URL / Command	Best For
Bash Manual	`man bash`	Complete reference
GNU Coreutils	`info coreutils`	find, tar, gzip docs
ShellCheck	shellcheck.net	Linting/validating scripts
Bash Guide	mywiki.wooledge.org/BashGuide	Deep dive, best practices
explainshell	explainshell.com	Break down complex commands
crontab.guru	crontab.guru	Visual cron schedule builder
Advanced Bash Guide	tldp.org/LDP/abs/html	Everything about bash
Bash Pitfalls	mywiki.wooledge.org/BashPitfalls	Common mistakes to avoid

---

Program Wrap-Up

What You've Mastered

```
SHELL SCRIPTING:                FILE MANAGEMENT:
================                ================
✅ Script structure & shebang    ✅ find by name, type, size
✅ Variables & expansion         ✅ find by time and permissions
✅ User input & validation       ✅ find -exec for batch operations
✅ Conditionals (if, case)       ✅ locate for instant searches
✅ Loops (for, while, until)     ✅ tar for archiving
✅ Functions & libraries         ✅ gzip / bzip2 / xz compression
✅ Arrays (indexed & assoc.)     ✅ zip for cross-platform sharing
✅ String & arithmetic ops       ✅ Complete backup workflows
✅ Error handling & traps        ✅ Log archiving & rotation
✅ Debugging techniques          ✅ Deployment packaging
✅ cron scheduling               ✅ Permission auditing
✅ at for one-time jobs
✅ systemd timers
✅ Production-ready logging
```

The Scripting Mindset

```
╔═══════════════════════════════════════════════════════════════╗
│                                                               │
│   IF YOU DO IT TWICE, SCRIPT IT                               │
│   → Repetition is the enemy of productivity                  │
│                                                               │
│   SET -EUO PIPEFAIL IN EVERY SCRIPT                           │
│   → Fail fast and loud — silent failures are worse           │
│                                                               │
│   QUOTE YOUR VARIABLES — ALWAYS                               │
│   → "$var" not $var. Filenames with spaces will break you.    │
│                                                               │
│   TEST YOUR CRON JOBS MANUALLY FIRST                          │
│   → Never add to crontab without testing ./script.sh first    │
│                                                               │
│   LOG EVERYTHING FROM CRON                                    │
│   → >> /var/log/myscript.log 2>&1 on every cron job           │
│                                                               │
│   find IS YOUR SWISS ARMY KNIFE                               │
│   → If you can describe it, find can find it                  │
│                                                               │
╚═══════════════════════════════════════════════════════════════╝
```

End of Bash Shell Scripting & File Management Training Guide

Written for beginners. Designed for automation. Built for production.